Personal Finance Management System

Analysis

Idea of the project

In daily life, people spend money on many different things including groceries, clothes, paying taxes, entertainment and etc. On top of that, some people are trying to invest money in different ways to receive passive income and having finance safety cushion. However, it might be hard for people to manage all spendings and income. That's why I decided to create a desktop application that will let users monitor their expenses and income in an efficient way.

There are already many finance managers developed and available for public use[1], so I decided to emphasize features of my future app that will stand out my app. After testing available finance trackers, I made the conclusion that most of them are aiming for single client use. Due to that reason, I set up a goal to create an app that will support multiple users. In order to reach this target, I prepared a list of functions that my app must have:

- Easy switching between users
- Log-in window after opening the program or after switching the user
- Register window for adding new users
- Editing user data

Another notice that I made was that some of app have unclear interfaces and too many details on the screen that make a confusion. So, I decided to develop user-friendly interface without overloaded design. I did research about UX/UI interfaces[2] and made a list of conclusions:

- **Maintain Visual Consistency:** Use uniform colors, fonts, and button styles throughout.
- **Minimize Cognitive Load:** Avoid clutter by hiding advanced features in menus.
- **Ensuring High Contrast Readability:** Make text stand out clearly against background colors.

After that, I know a key aspect of my project, but I still had a vague understanding of main functionality of my application. That why I decided to create a questionnaire and ask my friends who are students to complete it to get better understanding of desires of end user. I prepared such a questionnaire:

---

[1] https://emma-app.com/
https://snoop.app/

[2] https://www.figma.com/resource-library/ui-design-principles/
https://www.uxdesigninstitute.com/blog/10-user-interface-guidelines/
https://www.bbc.co.uk/supplying/working-with-us/user-experience

Questionnaire

1. How do you keep track of your money?

- A) I use a mobile app (like Monzo, Revolut, or a budgeting app)
- B) I use a spreadsheet (Excel/Google Sheets)
- C) I write it down on paper
- D) I just check my bank balance when I need to
- E) I don't track my money

2. What is your biggest challenge when it comes to managing your money as a student?

- A) Spending too much on food/takeaways
- B) Keeping track of subscriptions (Spotify, Netflix, gym, etc.)
- C) Saving up on big purchases (clothes, tech, holidays)
- D) Making my money lasts until the end of the month/term
- E) I don't really have any challenges

3. If you used a desktop finance tracker, what would be the most important feature for you?

- A) Seeing exactly where my money goes each month (spending by category)
- B) Setting monthly spending limits/budgets
- C) Being able to track multiple different accounts (e.g., Current Account and Savings)
- D) Having a very simple, fast way to add a new expense
- E) High security and password protection

4. How often would you realistically sit down at your computer to log your expenses?

- A) Every day
- B) A few times a week
- C) Once a week
- D) Once a month

- E) Rarely, I would forget

5. How do you prefer to see your financial summary?

- A) Visual graphs (like pie charts showing where my money went)
- B) A simple text list of my recent transactions
- C) A monthly calendar view showing when I spent money
- D) Just a big number showing my current total balance
- E) It doesn't matter to me

6. What is the number one reason you would stop using a finance tracking app?

- A) It takes too much time/effort to enter a transaction
- B) The screen is too cluttered and confusing to look at
- C) It doesn't give me any useful information or graphs
- D) It crashes or has bugs
- E) I just lose motivation to track my money

Results of Questionnaire

	A	B	C	D	E
Q1	15%	**35%**	10%	10%	**30%**
Q2	**25%**	**28%**	12%	**27%**	8%
Q3	10%	12%	**38%**	**35%**	5%
Q4	15%	**30%**	15%	5%	**35%**
Q5	12%	**42%**	8%	18%	20%
Q6	15%	**32%**	10%	8%	**35%**

This result lets me draw several conclusions. First of all, most people don't either use budget apps or use excel/google sheet for that purpose. Also, I understood that people would like to have simple apps that support multiple accounts. Finally, people would likely stop using finance tracking apps if they have unclear design.

After analyzing the questionnaire, I got the understanding of functions that I will have to implement to satisfy end user.

Choosing Programming Language for the Project

Initially, I had 3 languages that I considered for the project, namely Java, C# and Python. I researched about them[3] and created a table with pros and cons for each language to choose the optimal language.

	Java	C#	Python
Pros	• Previous experience of developing Desktop apps • Powerful library JavaFX for Desktop Apps • Strong OOP principles and Strongly Typed	• Some knowledge from A-Level • Libraries tailored for Windows • Powerful OOP	• Beginner friendly • Rapid development
Cons	• JVM might require more resources • More line codes required compared to Python	• Complex Microsoft ecosystem • Slow Performance	• GUI limitation • Not Strongly Typed • No Previous experience

After weighing the pros and cons, I chose **Java** because I had experience of developing desktop apps via java before. Furthermore, it provides good structure of the project and supports OOP. Finally, this decision solved the question of choosing GUI library because I already knew that there is JavaFX library that ideally suits the aims of my project.

Choosing Database for the Project

My next dilemma was about databases that I should use for storing all required information. I have done similar research[4] and created the table.

	SQLite	MySQL	MongoDB
Pros	• Doesn't require server • Can be stored in single file	• Scalable: Can handle massive amount of data • Advanced Security	• Flexible Schema • Good Performance for Unstructured data
Cons	• Doesn't support Concurrency	• Complex Setup • Request more recourses	• Doesn't support joins

[3] http://naukri.com/code360/library/java-vs-python-and-c
[4] https://skynix.co/resources/pros-and-cons-mysql-postgresql-sqlite-mongodb

	• Lack of Security		• Redundancy of Data

After a deep comparison, I settled on **SQLite** because of its simplicity and ease of setting up. It has certain drawbacks such as lack of concurrency which might be a problem in case of scaling and improving the project, but due to standard SQL, it is not a problem to migrate to other relational databases such as MySQL. So far, it meets all my requirements.

Build tool

My java project will need a build tool in order to be able to use JavaFX and SQLite in my project. For that purpose, I chose Apache Maven Framework[5]. Apache Maven is a build automation and project management utility commonly used in Java development. It simplifies the build process, dependency management, and documentation of projects following the "convention over configuration" principle to reduce setup time. Also, it promotes a uniform directory structure for the project. I chose "Standard Maven Directory Structure"[6] for my project. As a result, my project will organize in a such way:

- src/main/java/ (for all Java source code)
- src/main/resources/ (for non-code assets like FXML files, CSS, SQL scripts, and images)
- target/ (where Maven puts the compiled .class files and the final .jar file)

Also, there will be a configuration file called pom.xml. In this file, I will add dependencies for JavaFX and driver for SQLite, so I can use them in my project.

To find needed dependencies, I used website Maven Repository[7] where are stored most of dependencies. There is a code that I will have to add to pom.xml config to add JavaFX and SQLite driver:

JavaFx:

```
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>${javafx.version}</version>
</dependency>
<dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-fxml</artifactId>
    <version>${javafx.version}</version>
</dependency>
```

[5] https://www.browserstack.com/guide/what-is-maven-in-java
[6] https://www.baeldung.com/maven-directory-structure
[7] https://mvnrepository.com/

SQLite:

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.49.1.0</version>
</dependency>
```

Objectives broken down into measurable success criteria:

Main objective	Explanation	Success Criteria
1. Secure User Authentication	Implement secure logging in in a separate window with applying hash function to the password for comparison with hash stored in database	a) Window with user and password fields b) Buttons “Cancel” and “Login” c) Applying hash function to password d) Comparing hash of entered password with hash stored in database
2. Hash Utility	Implement hash function that will be used for hashing password	a) hash function algorithm that will return hash of the string
3. Database Implementation	Design normalized database that will contain all data needed for the program including users, transactions, accounts and categories	a) Connecting to Database b) Checking that database contains all the tables needed. c) Normalized form d) Closing connection after closing the app
4. CRUD operations with Database	Writing all methods and sql queries needed for proper work of app	a) Methods and queries for retrieving, updating, deleting and adding transactions b) Methods and queries for retrieving, updating,

		deleting and adding accounts c) Methods and queries for retrieving, updating, deleting and adding categories d) Methods and queries for retrieving, updating, deleting and adding users
5. Session Management	Keeping track of current user and not letting using app until user is logged-in	a) Tracking current user b) Change current user during switching. c) Freezing app until user is not logged-in
6. Main window with tabs	Implement main window that will contain all tabs and will have sidebar with current user and quick action such as adding new income and transaction	a) Main window that shows current tab b) Sidebar with all tabs c) Quick action to add income/expense d) Switch user button e) Manage users button
7. Dashboard tab	Implement dashboard that will contain Monthly income and expenses. Also, it will contain recent transactions	a) Field that will show monthly total monthly income and expense based on transactions b) Table with recent transactions
8. Transaction tab	Implement tab that will contain table with all transactions and add filters fields for searching	a) Table with columns: type, category, amount, date, user, account, description. b) Filters such as type, category, amount

		date, user and account c) Search by description
9. Account tab	Implement table with all accounts. Also, add search and add “Add New Account” button	a) Table with all existing accounts b) Search field and corresponding button c) “Create new Account” button
10. Category tab	Implement table with all categories. Also, add search and add “Add New Category” button	a) Table with all existing categories b) Search field and corresponding button c) “Create new Category” button
11. Adding new Transaction	Implement windows for adding Income and Expanse transactions. Window must contain fields such as amount and description. Also, user must be able to choose account and category	a) Window for adding income/expanse. b) Account combobox c) Amount field d) Category combobox e) Description field
12. Deleting and Editing a Transaction	User must be able to delete or edit some data of transaction by clicking on certain transaction in the table in Transaction tab	a) Window for deleting or editing transaction. b) In case of editing, another window for changing amount or description
13. Filtering and searching transaction by parameters	Implement all methods and sql queries for searching for a certain transaction via available filters	a) Sql query is modified based on used filters b) Table shows only transactions that meet all criteria
14. Creating new Account	Implement window that will open when user press “Create new Account” button. Window must contain account type combobox, and account	a) Opening separate window b) Combobox for account type c) Account name field

	name filed. Also, user must be able to add as many users to account as they want	d) List of users that can be added
15. Deleting and Editing an Account	User must be able to delete or edit some data of account by clicking on certain account in the table in Account tab	a) Window for deleting or editing account. b) In case of editing, another window for changing type, name or users
16. Searching needed Account	Implement all methods and sql queries for searching for a certain account	a) Sql query is modified based on used filters b) Table shows only accounts that meet all criteria
17. Creating new Category	Implement window that will open when user press “Create new Category” button. Window must contain category type, category name and description	a) Opening separate window b) Combobox for category type c) Category name field d) Description field
18. Deleting and Editing a Category	User must be able to delete or edit some data of category by clicking on certain category in the table in Category tab	a) Window for deleting or editing category. b) In case of editing, another window for changing type, name or description
19. Searching needed Category	Implement all methods and sql queries for searching for a certain account	a) Sql query is modified based on used filters b) Table shows only categories that meet all criteria
20. Switching between Users	Implement switching between users. Every time	a) List of users for switching

	after switching user must log-in	b) Log-in window that will appear after switching users
21. Registering new Users	Implement window for adding new users. There must be username and password fields.	a) Window for adding new users b) Username and password field c) Hash function must be applied to password before storing in database
22. Deleting and Editing details of Users	User must be able to delete other user or edit username	a) Separate window for deleting or editing users. b) In case of editing, separate window must appear with username field
23. Input Validation and Error Handling	The system must handle invalid user inputs gracefully without crashing.	a) The system rejects alphabetical characters entered into the numerical "Amount" field. b) The system prevents submission of a transaction if mandatory fields (like Account or Category) are left blank, displaying an alert dialog instead.

Design

Patterns

The first thing that I started thinking about is an Architectural Pattern. For that purpose, I chose MVC(Mode-View-Controller pattern)[8] because I already had experience of using it and thought that it would suit well for this project.

This pattern consists of 3 components:

- Model - Manages application data and business logic.
- View - Handles the user interface and presentation of data
- Controller - Processes user input and coordinates between Model and View.

This strict separation of concerns ensures the code is highly modular, making it significantly easier to debug, test, and maintain without accidentally breaking the visual layout while changing backend logic.

In my project, **Model** will consist of Java classes such as User, Account and Transaction. **View** will contain all UI components and particularly all visual layouts. Finally, **Controller** will contain classes that handle interaction between visual components and model such as handling buttons, updating information on app and sending request to model classes.

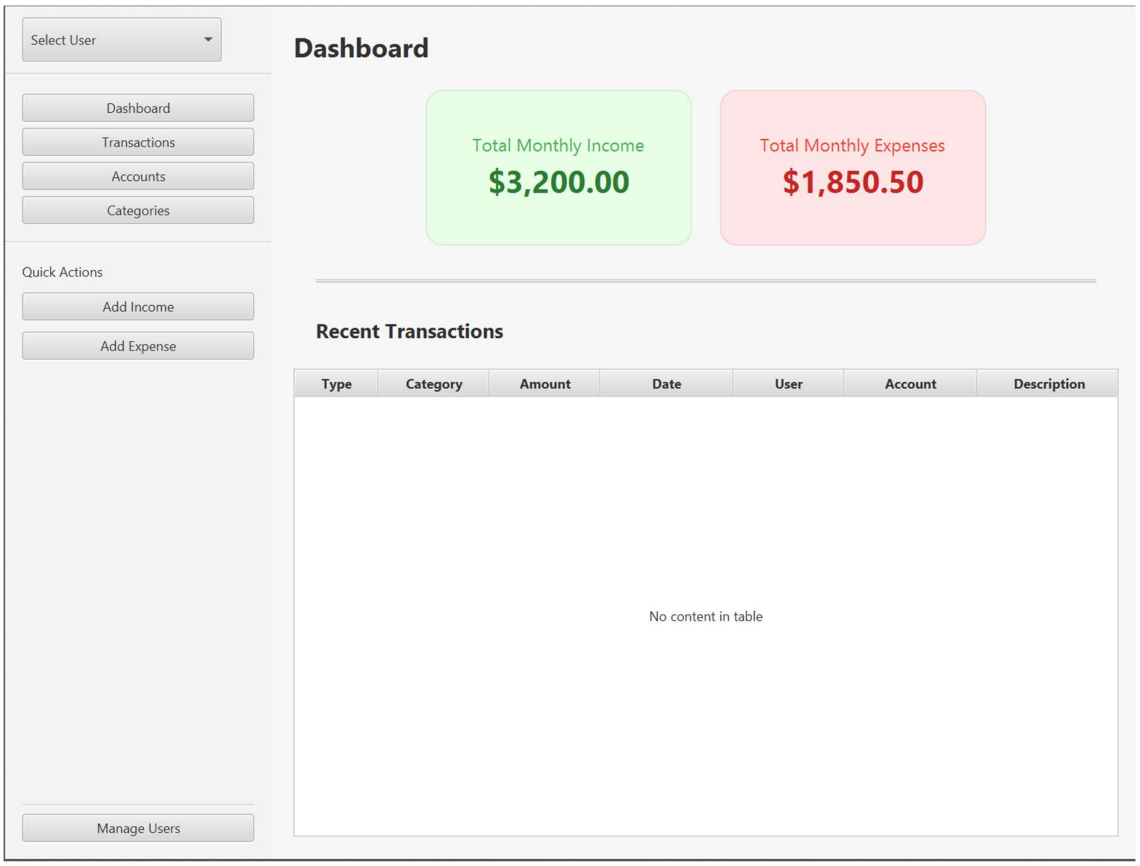
In addition to MVC, I decided to use DAO(Data Access Object)[9] pattern because I understand that my program will contain many sql queries, so I want to avoid mess and have all database related stuff in one place. Basically, DAO abstracts and encapsulates all access to the underlying SQLite database. As a result, all database operations will be delegated to specific DAO classes, such as TransactionsDAO and UserDAO.

On top of that, in order to support multiple user operation, I must keep track of the current user. For that reason, I created UserSession class that contains information about current user. Also, this class lets change the current user when users are switched. My program must contain only one instance of UserSession class and because of that I implemented a Singleton Pattern[10].

Interface

[8] https://www.geeksforgeeks.org/system-design/mvc-design-pattern/
[9] https://www.geeksforgeeks.org/system-design/data-access-object-pattern/
[10] https://www.baeldung.com/java-singleton

For creating the interface of the project, I used the tool called JavaFX Scene Builder[11]. It is a visual layout app that lets users quickly design JavaFX application user interfaces, without coding. Via this tool, I created all fxml[12] layouts that I need. I decided to have a main window and several tabs between which the user can switch. For **main window**, I have designed such an interface:



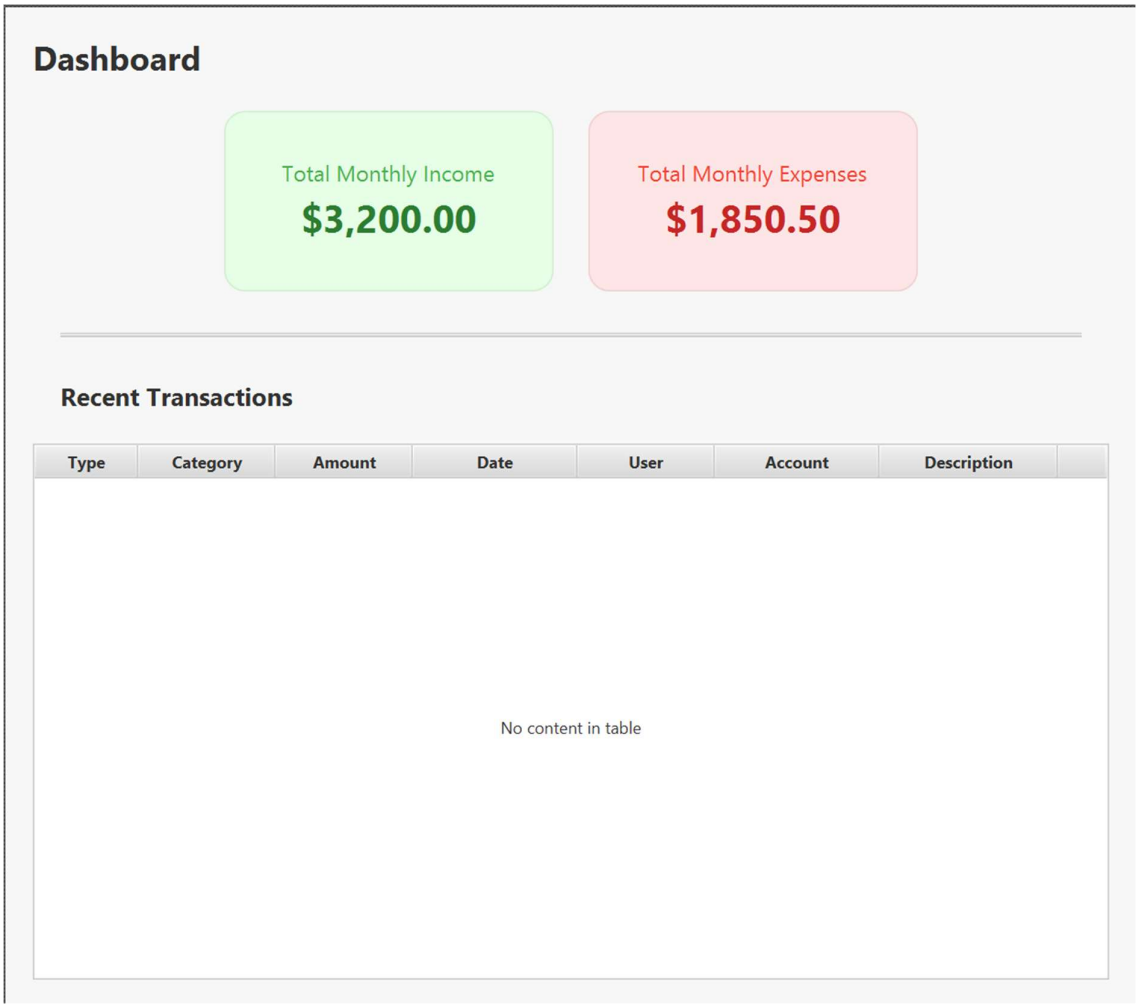
Main window

The area to the right of the sidebar is imported from other fxml file that describe tabs. By replacing the path to the imported fxml layout, users will be able to see other tabs.

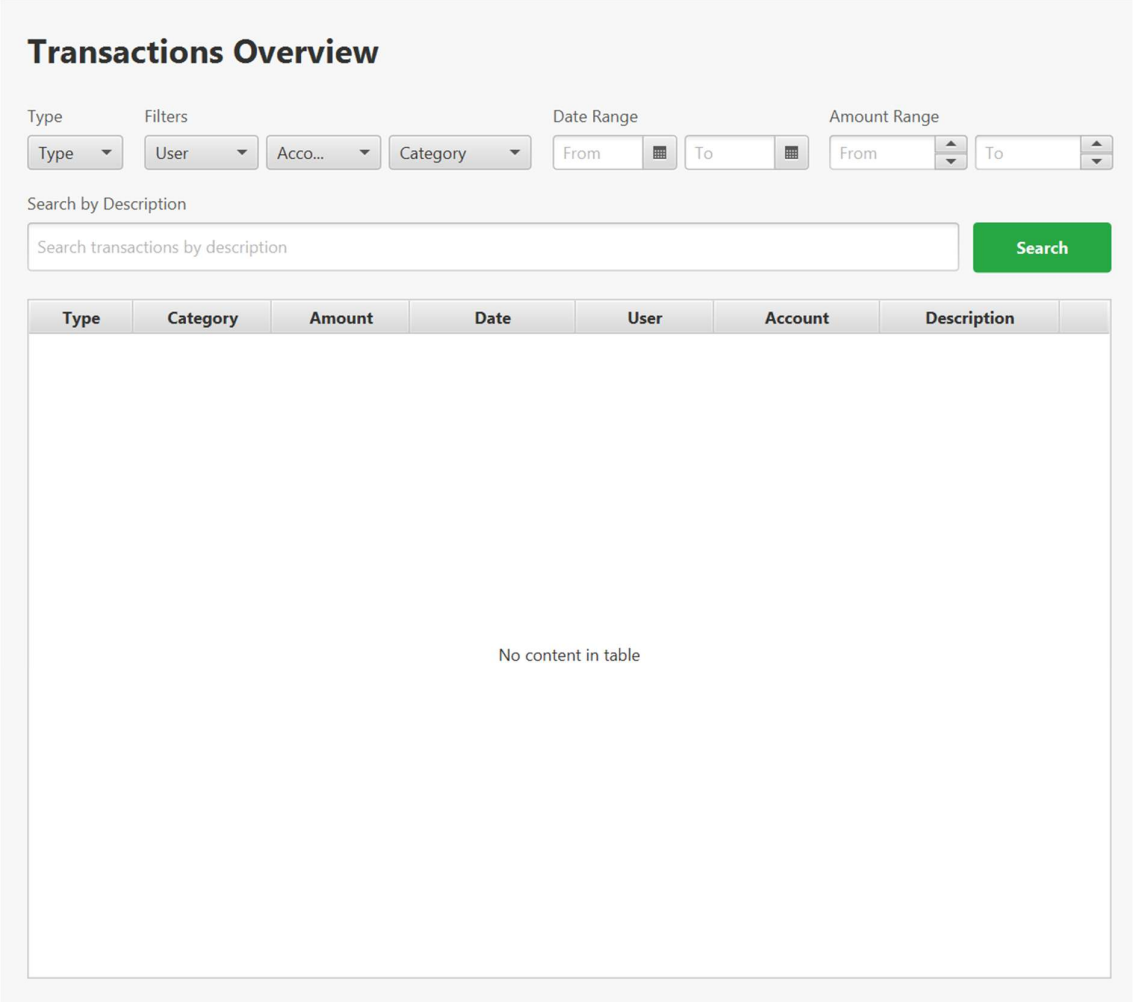
I have designed tabs for Dashboard, Transactions, Accounts and Categories:

[11] https://www.oracle.com/java/technologies/javase/javafxscenebuilder-info.html#:~:text=JavaFX%20Scene%20Builder%20is%20a,application%20user%20interfaces%2C%20without%20coding

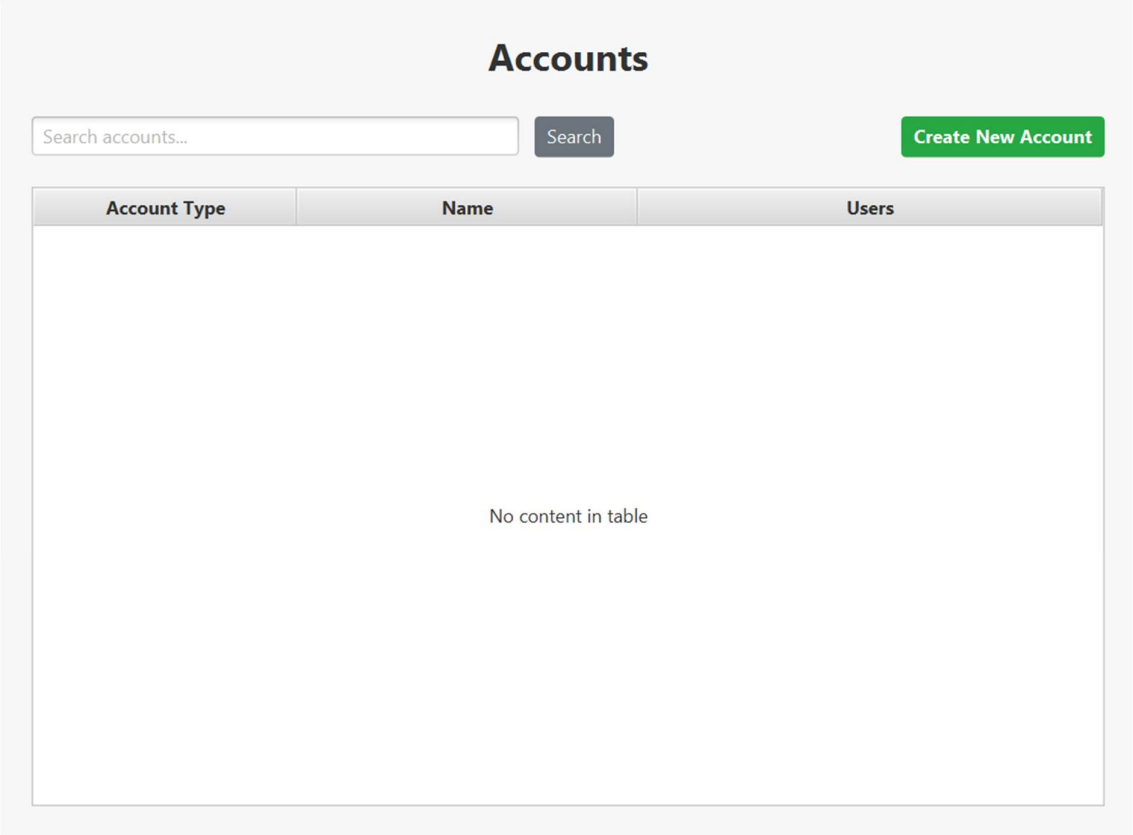
[12] https://jenkov.com/tutorials/javafx/fxml.html



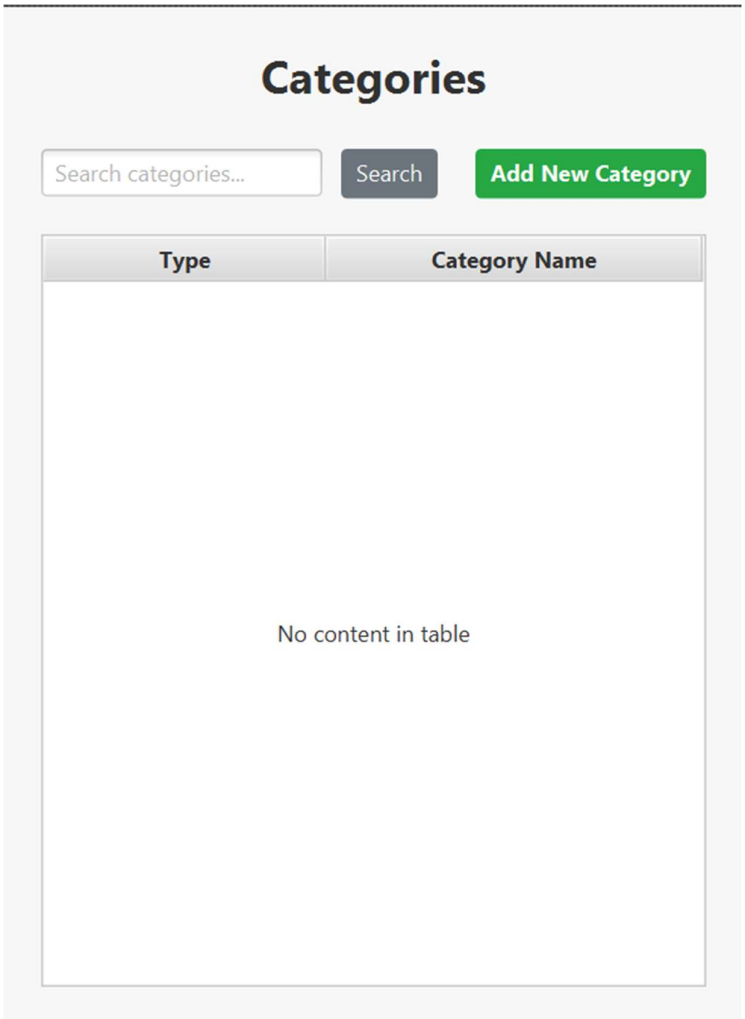
Dashboard tab



Transactions tab

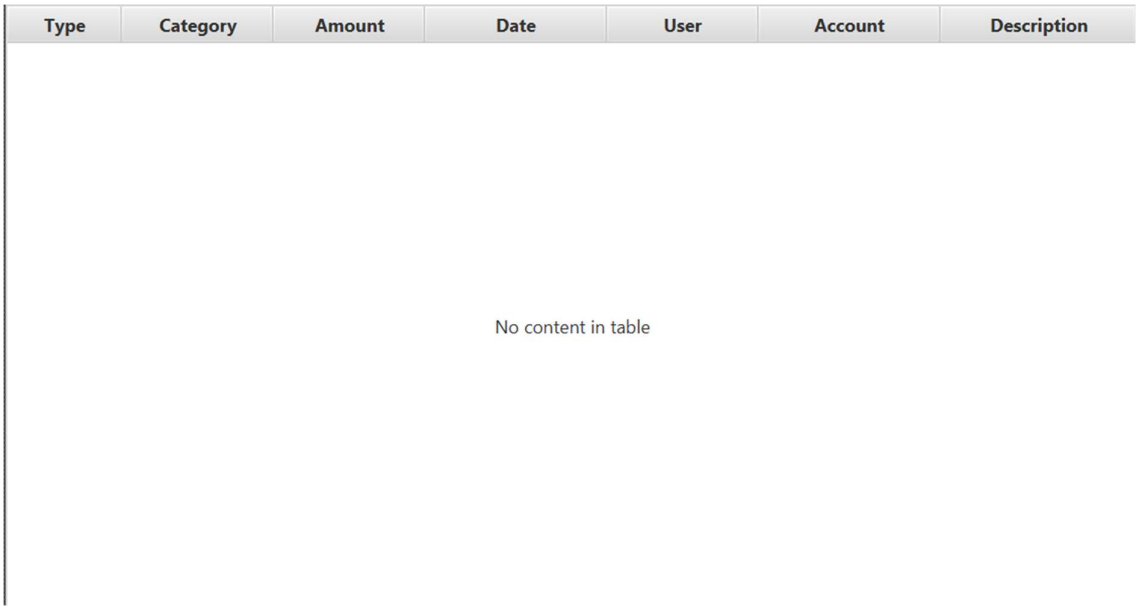


Accounts tab



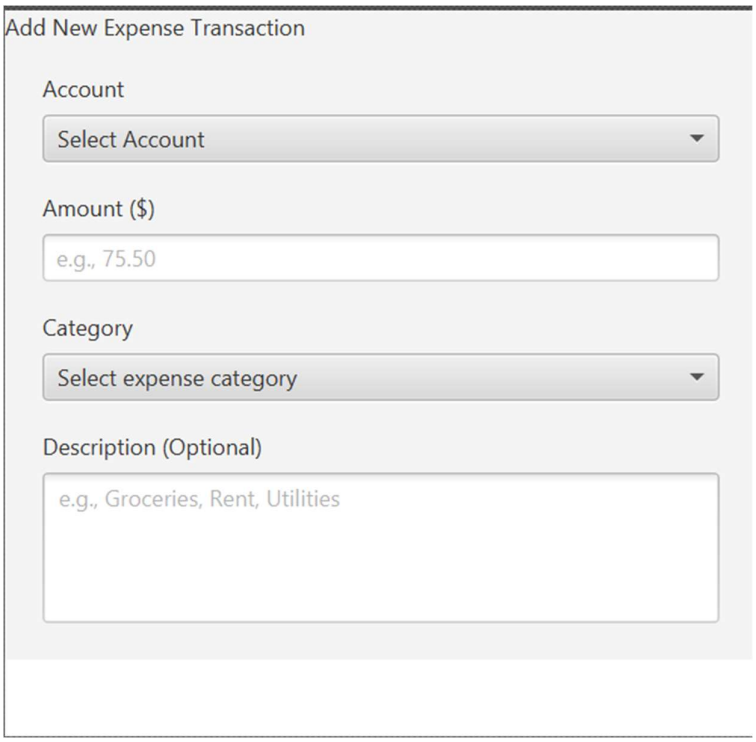
Categories tab

Also, I have created a separate fxml layout for transactionTable that will be imported into Recent Transactions section in Dashboard and in Transactions tab.

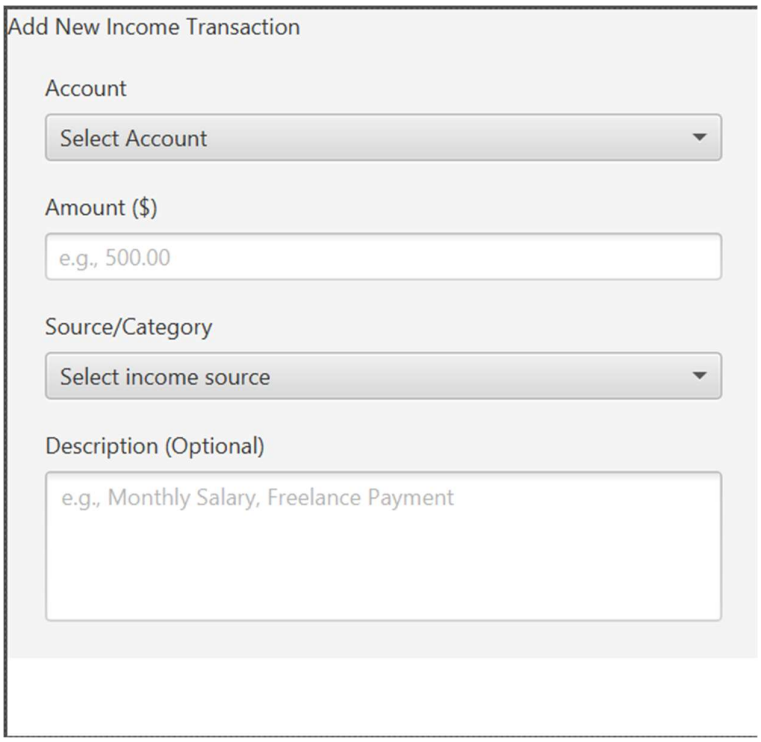


Transactions Table

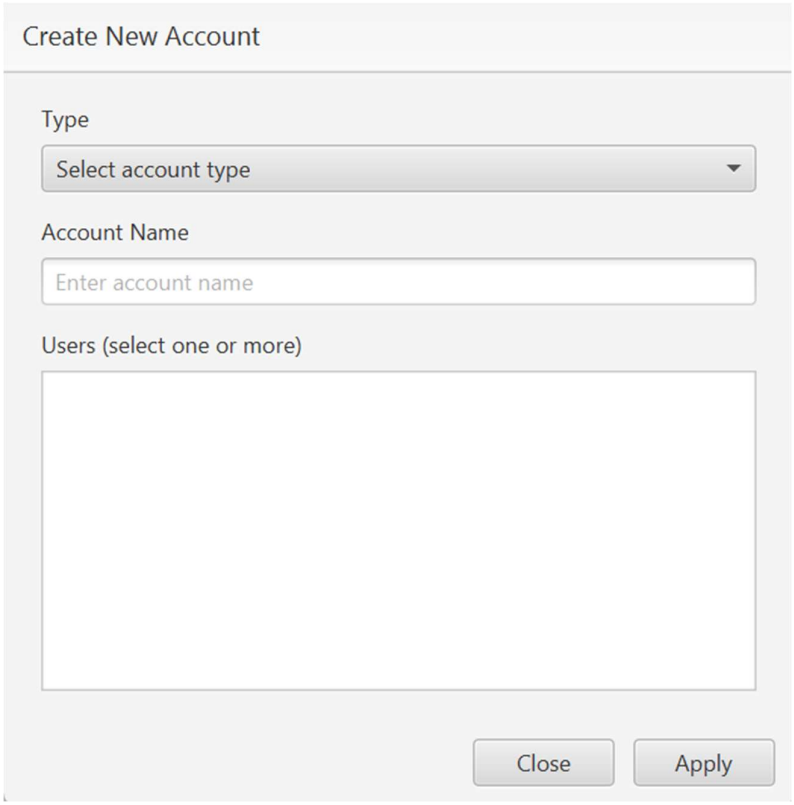
After that I started designing dialog windows which will let user do many things such as logging-in, adding income/expense, editing transactions and so on. In total, I have designed 7 dialogs, namely Adding Expense, Adding Income, Adding Account, Adding Category, Log In, Manage Users, and User Registration:



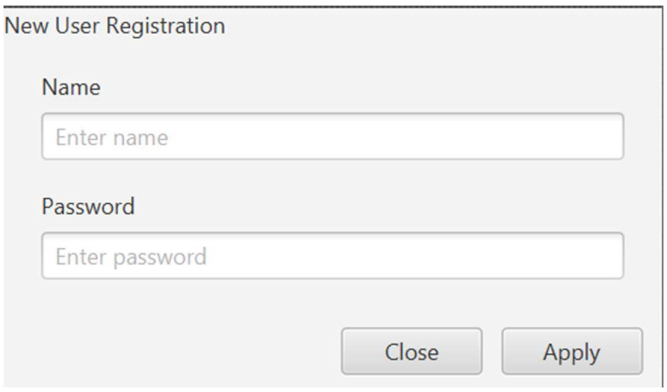
Add Expense



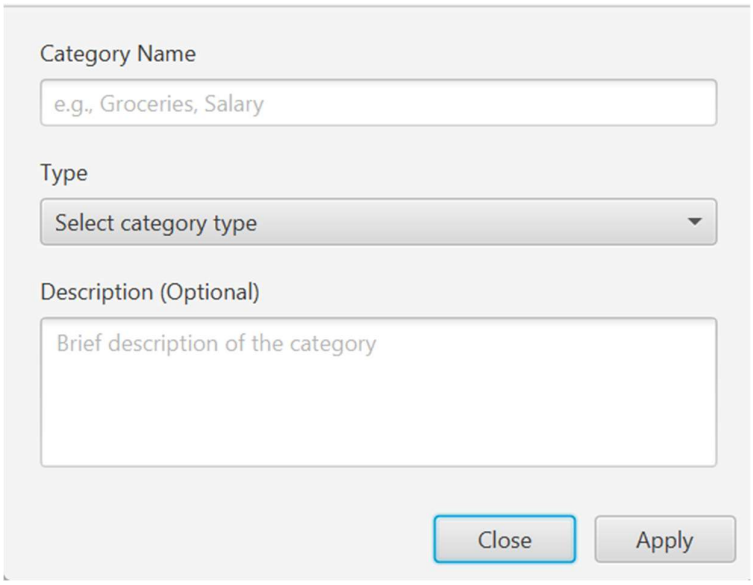
Add Income



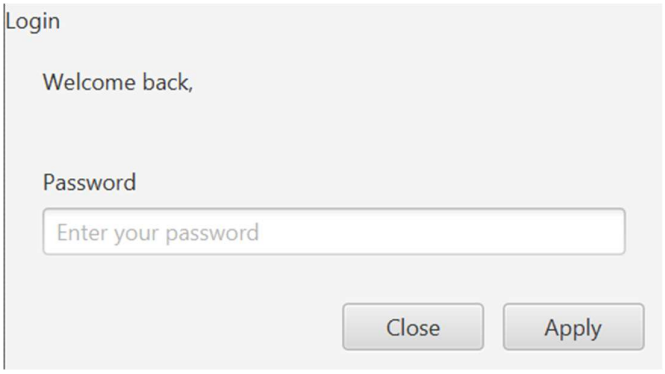
Create Account



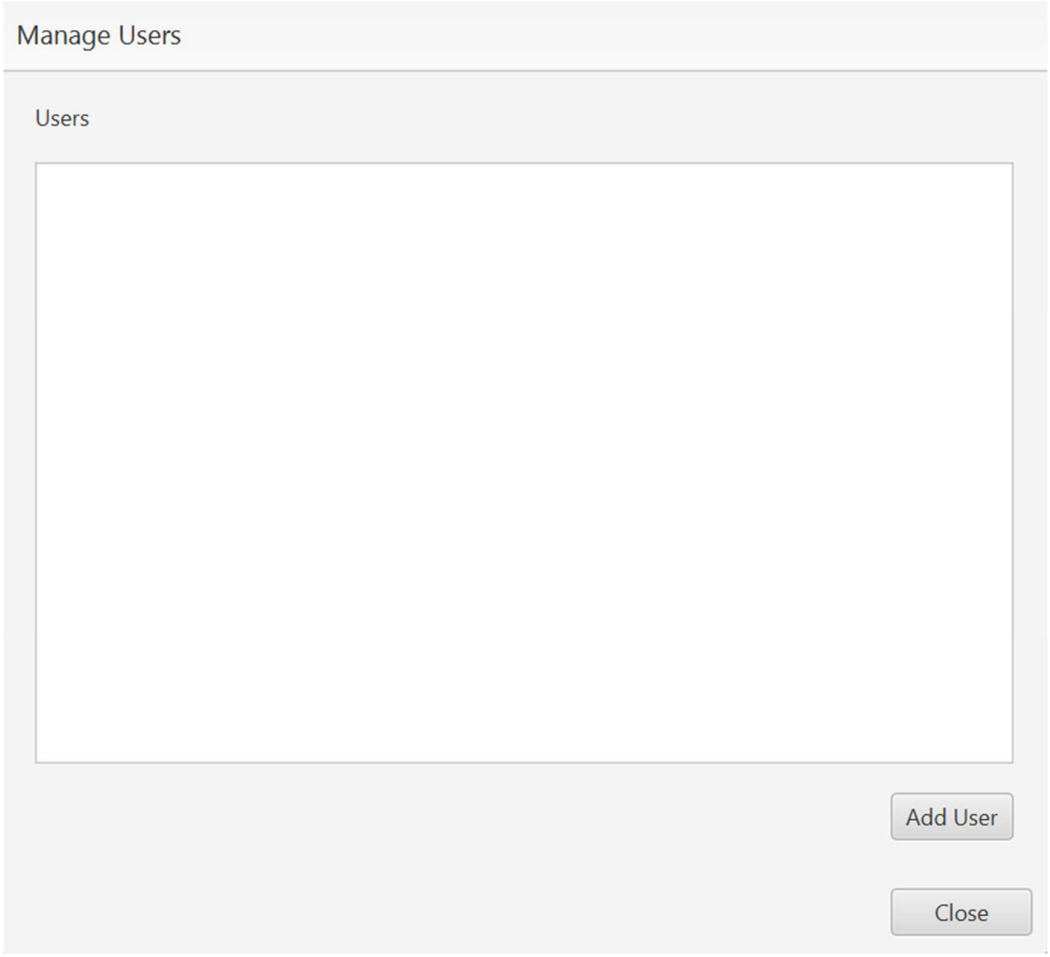
User Registration



Create Category



Login



Manage Users

Project structure

I started creating a structure of my project by exploring JavaFX[13] library that will handle all UI components as well as controllers. JavaFX is a Java library and GUI toolkit for developing Rich Internet Applications (RIA), web applications, and desktop applications. Its main advantage is cross-platform compatibility, running on Windows, Linux, iOS android, desktops, web, TVs and tablets.

The JavaFX Architecture

JavaFX application has two main containers:

1) *Stage* - the main container and the entry point of the application. It represents the main window and passed as an argument of the *start()* method.

[13] https://www.geeksforgeeks.org/java/javafx-tutorial/
https://www.baeldung.com/javafx

2) *Scene* – a container for holding the UI elements, such as Image Views, Buttons, Grids, TextBoxes.

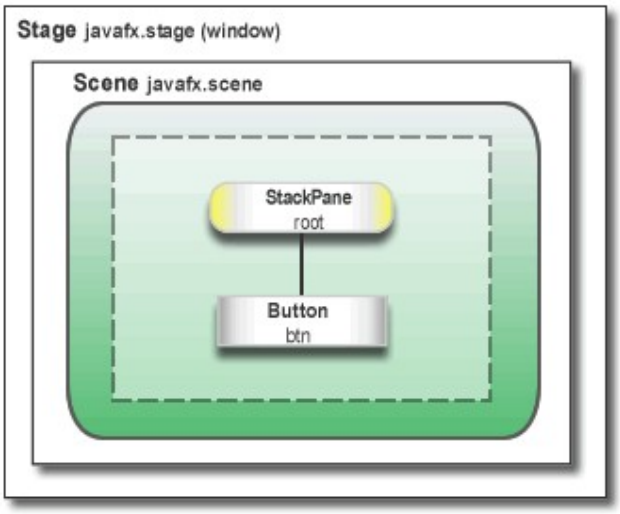


Diagram from baeldung.com

The content of a Scene is built using a Scene Graph—a hierarchical tree of "Nodes". Scenes can be replaced or switched.

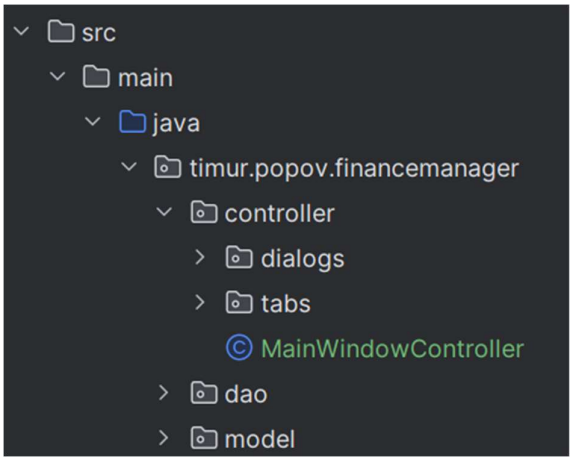
Also, JavaFx separates Design and Logic. All visual layouts are stored in FXML(modified XML)[14] files. Each fmxl file is linked to Java Controller class that handles all interaction of UI(User Interface) such as pressing buttons, editing UI components and sending requests to the Model classes.

After understanding the architecture of JavaFX, I started designing the dictionary structure of my project. First of all, I created packages where will be located my Java Classas. As I already mentioned, in MVC architecture, all java classes must be stored in pakage src/java/... So I started designing from that place. I created several packages, namely:

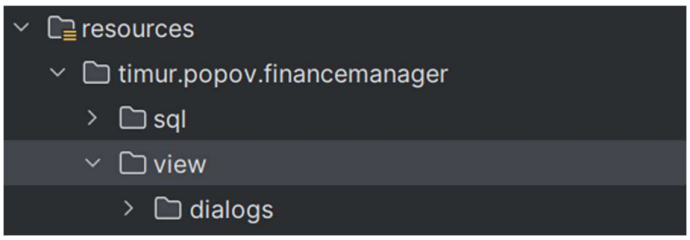
- **controller** - all controllers linked to fxml files
- **dao** - java classes that will handle database queries and all related stuff
- **model** - core java classes such as User, Account, Category and Transaction

Furthermore, I add packages **dialogs** and **tabs** into controller which will contain corresponding classes. As a result, I got such a structure:

[14] https://en.wikipedia.org/wiki/FXML



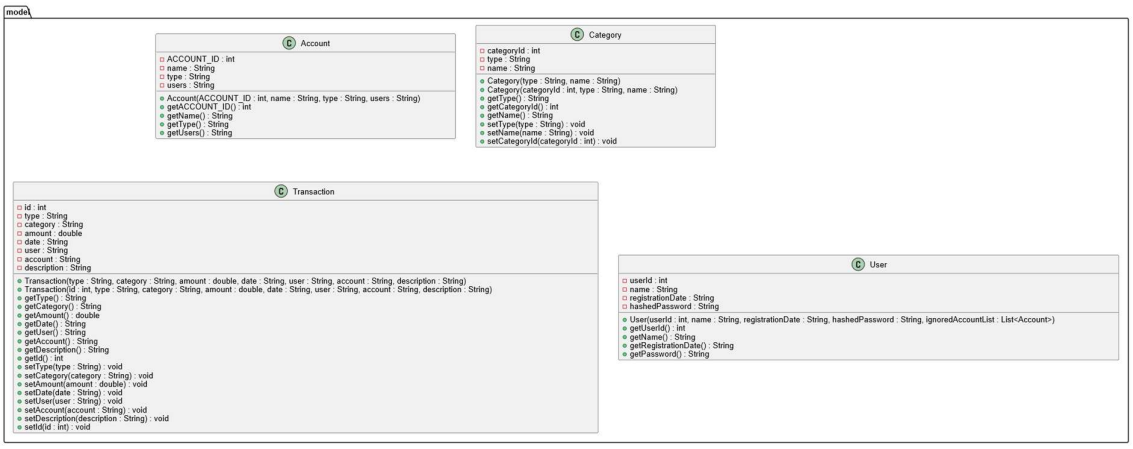
After that, I started working on folder “resources” that will contain all fxml files as well as sql script file for creating tables in database and styles.css for UI. I created folder **view** for FXML layouts and **sql** for scripts. On top of that, inside “view” folder, I created a subfolder for dialogs.



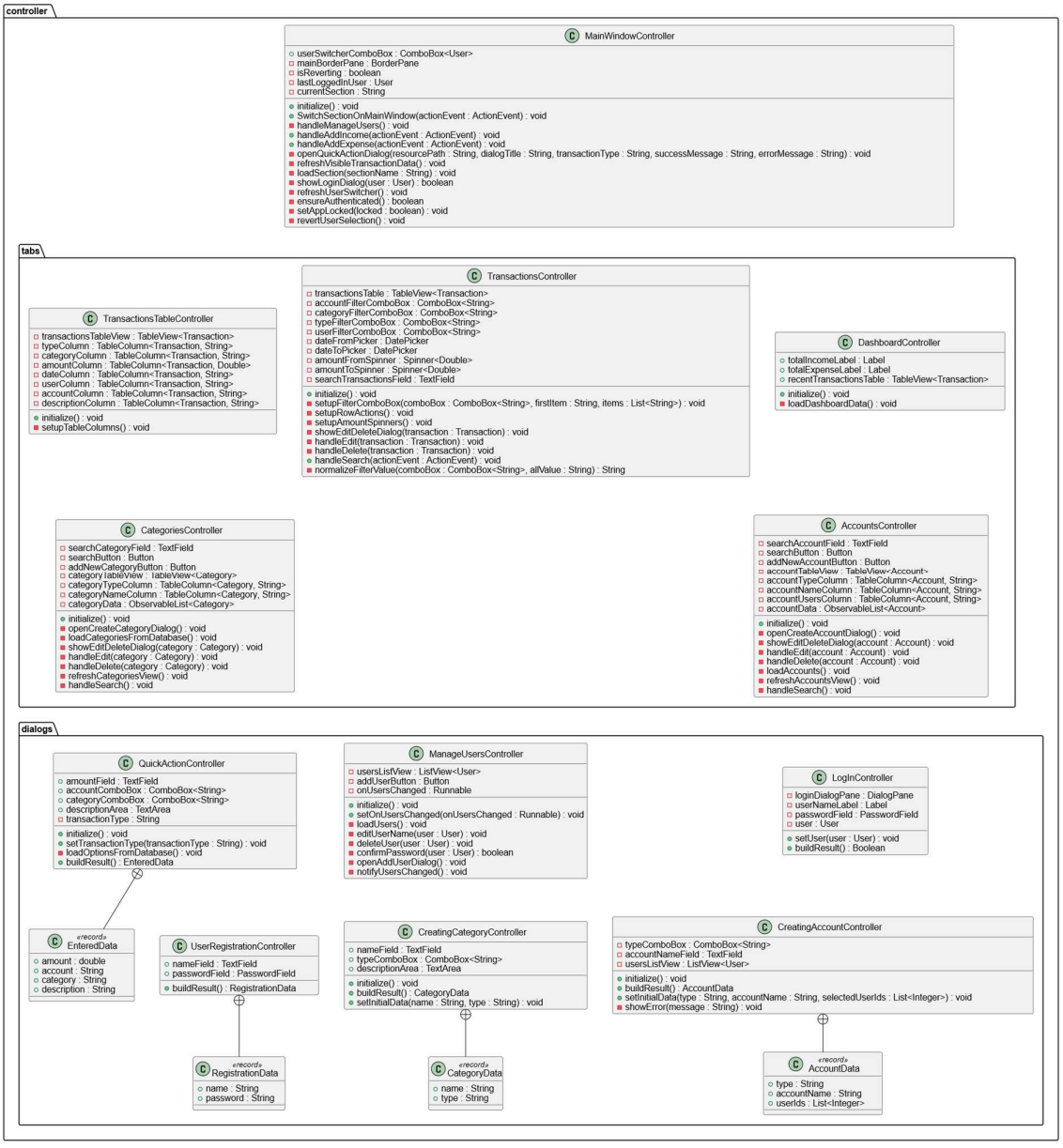
Java classes

Before starting coding, I declared all classes that I will need. For that purpose, I used UML diagrams to keep it clear.

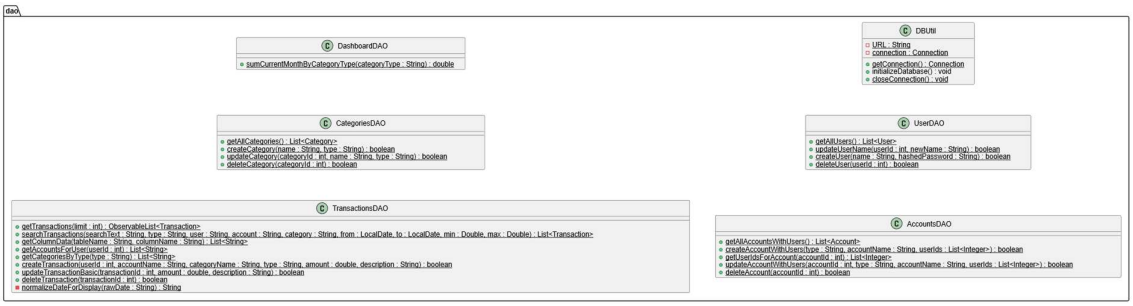
Model:



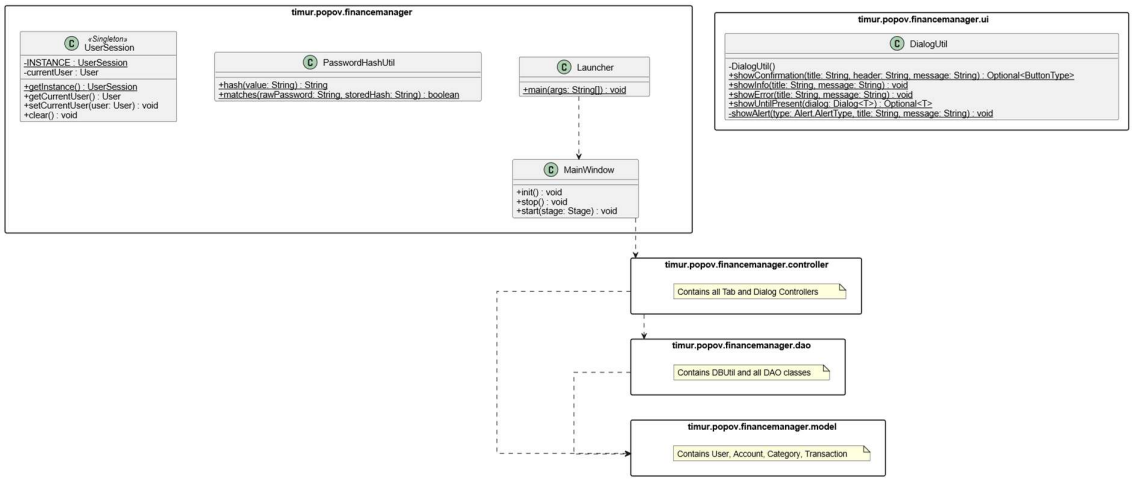
Controller:


controller
MainWindowController
- userSwitcherComboBox : ComboBox<User>
- mainBorderPane : BorderPane
- isReverting : boolean
- lastLoggedInUser : User
- currentSection : String
+ initialize() : void
+ SwitchSectionOnMainWindow(actionEvent : ActionEvent) : void
- handleManageUsers() : void
+ handleAddIncome(actionEvent : ActionEvent) : void
+ handleAddExpense(actionEvent : ActionEvent) : void
- openQuickActionDialog(resourcePath : String, dialogTitle : String, transactionType : String, successMessage : String, errorMessage : String) : void
- refreshVisibleTransactionData() : void
- loadSection(sectionName : String) : void
- showLoginDialog(user : User) : boolean
- refreshUserSwitcher() : void
- ensureAuthenticated() : boolean
- setAppLocked(locked : boolean) : void
- revertUserSelection() : void
tabs
TransactionsTableController
- transactionsTableView : TableView<Transaction>
- typeColumn : TableColumn<Transaction, String>
- categoryColumn : TableColumn<Transaction, String>
- amountColumn : TableColumn<Transaction, Double>
- dateColumn : TableColumn<Transaction, String>
- userColumn : TableColumn<Transaction, String>
- accountColumn : TableColumn<Transaction, String>
- descriptionColumn : TableColumn<Transaction, String>
+ initialize() : void
- setupTableColumns() : void
TransactionsController
- transactionsTable : TableView<Transaction>
- accountFilterComboBox : ComboBox<String>
- categoryFilterComboBox : ComboBox<String>
- typeFilterComboBox : ComboBox<String>
- userFilterComboBox : ComboBox<String>
- dateFromPicker : DatePicker
- dateToPicker : DatePicker
- amountFromSpinner : Spinner<Double>
- amountToSpinner : Spinner<Double>
- searchTransactionsField : TextField
+ initialize() : void
- setupFilterComboBox(comboBox : ComboBox<String>, firstItem : String, items : List<String>) : void
- setupRowActions() : void
- setupAmountSpinners() : void
- showEditDeleteDialog(transaction : Transaction) : void
- handleEdit(transaction : Transaction) : void
- handleDelete(transaction : Transaction) : void
+ handleSearch(actionEvent : ActionEvent) : void
- normalizeFilterValue(comboBox : ComboBox<String>, allValue : String) : String
DashboardController
- totalIncomeLabel : Label
- totalExpenseLabel : Label
- recentTransactionsTable : TableView<Transaction>
+ initialize() : void
- loadDashboardData() : void
CategoriesController
- searchCategoryField : TextField
- searchButton : Button
- addNewCategoryButton : Button
- categoryTableView : TableView<Category>
- categoryTypeColumn : TableColumn<Category, String>
- categoryNameColumn : TableColumn<Category, String>
- categoryData : ObservableList<Category>
+ initialize() : void
- openCreateCategoryDialog() : void
- loadCategoriesFromDatabase() : void
- showEditDeleteDialog(category : Category) : void
- handleEdit(category : Category) : void
- handleDelete(category : Category) : void
- refreshCategoriesView() : void
- handleSearch() : void
AccountsController
- searchAccountField : TextField
- searchButton : Button
- addNewAccountButton : Button
- accountTableView : TableView<Account>
- accountTypeColumn : TableColumn<Account, String>
- accountNameColumn : TableColumn<Account, String>
- accountUsersColumn : TableColumn<Account, String>
- accountData : ObservableList<Account>
+ initialize() : void
- openCreateAccountDialog() : void
- showEditDeleteDialog(account : Account) : void
- handleEdit(account : Account) : void
- handleDelete(account : Account) : void
- loadAccounts() : void
- refreshAccountsView() : void
- handleSearch() : void
dialogs
QuickActionController
- amountField : TextField
- accountComboBox : ComboBox<String>
- categoryComboBox : ComboBox<String>
- descriptionArea : TextArea
- transactionType : String
+ initialize() : void
+ setTransactionType(transactionType : String) : void
- loadOptionsFromDatabase() : void
+ buildResult() : EnteredData
«record»
EnteredData
+ amount : double
+ account : String
+ category : String
+ description : String
UserRegistrationController
- nameField : TextField
- passwordField : PasswordField
+ buildResult() : RegistrationData
«record»
RegistrationData
+ name : String
+ password : String
ManageUsersController
- usersListView : ListView<User>
- addUserButton : Button
- onUsersChanged : Runnable
+ initialize() : void
+ setOnUsersChanged(onUsersChanged : Runnable) : void
- loadUsers() : void
- editUserName(user : User) : void
- deleteUser(user : User) : void
- confirmPassword(user : User) : boolean
- openAddUserDialog() : void
- notifyUsersChanged() : void
CreatingCategoryController
- nameField : TextField
- typeComboBox : ComboBox<String>
- descriptionArea : TextArea
+ initialize() : void
+ buildResult() : CategoryData
+ setInitialData(name : String, type : String) : void
«record»
CategoryData
+ name : String
+ type : String
LogInController
- loginDialogPane : DialogPane
- userNameLabel : Label
- passwordField : PasswordField
- user : User
+ setUser(user : User) : void
+ buildResult() : Boolean
CreatingAccountController
- typeComboBox : ComboBox<String>
- accountNameField : TextField
- usersListView : ListView<User>
+ initialize() : void
+ buildResult() : AccountData
+ setInitialData(type : String, accountName : String, selectedUserIds : List<Integer>) : void
- showError(message : String) : void
«record»
AccountData
+ type : String
+ accountName : String
+ userIds : List<Integer>

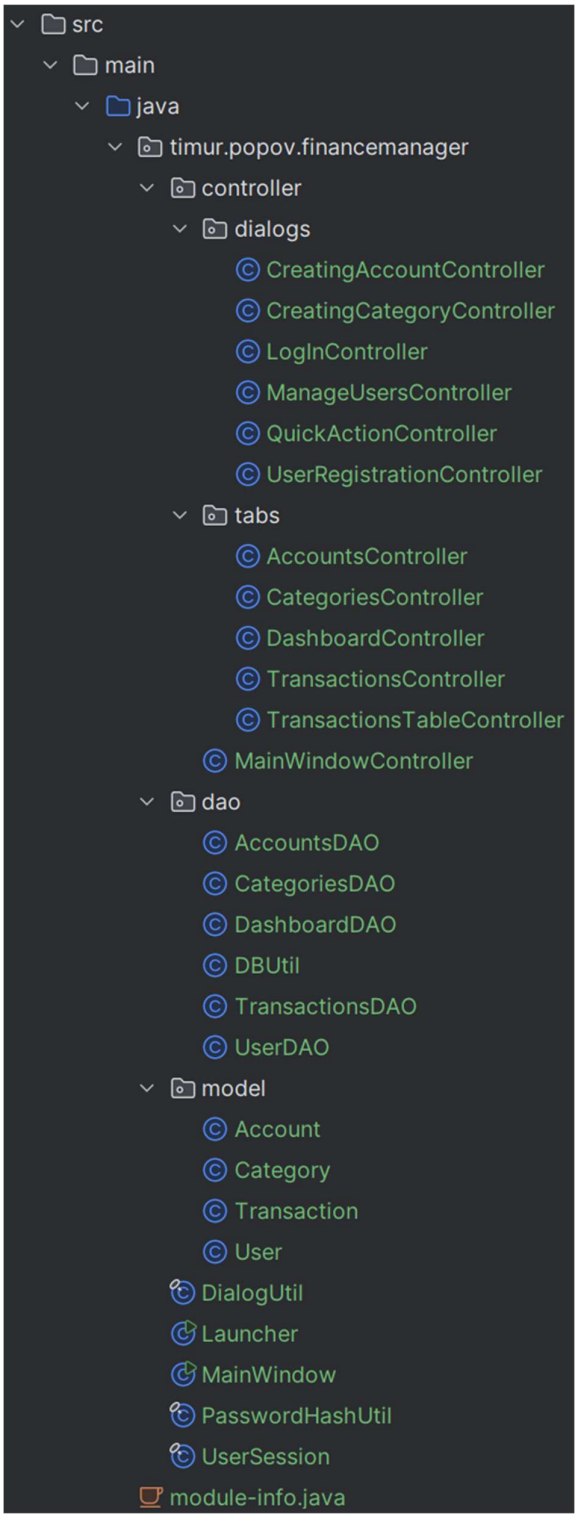
DAO:

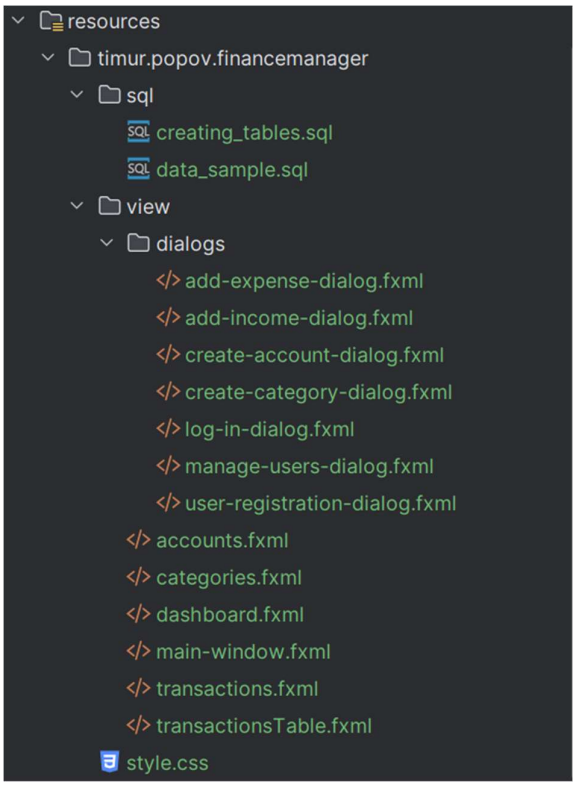


General diagram of the project:



Final design of the project:





Database

Database is a key component in my application because it will store all users' data and will be used for retrieving them. I want my database to be efficient, consistent and accurate, so I have to use Database Normalization[15]. Normalization helps to get rid of redundancy and ensures that data are consistent. I am aiming to get Third Normal Form (3NF). In order to do that, my tables must contain only atomic data and have no repeats. Also, tables must have no partial dependencies and each non-prime attribute must dependent solely on the primary key.

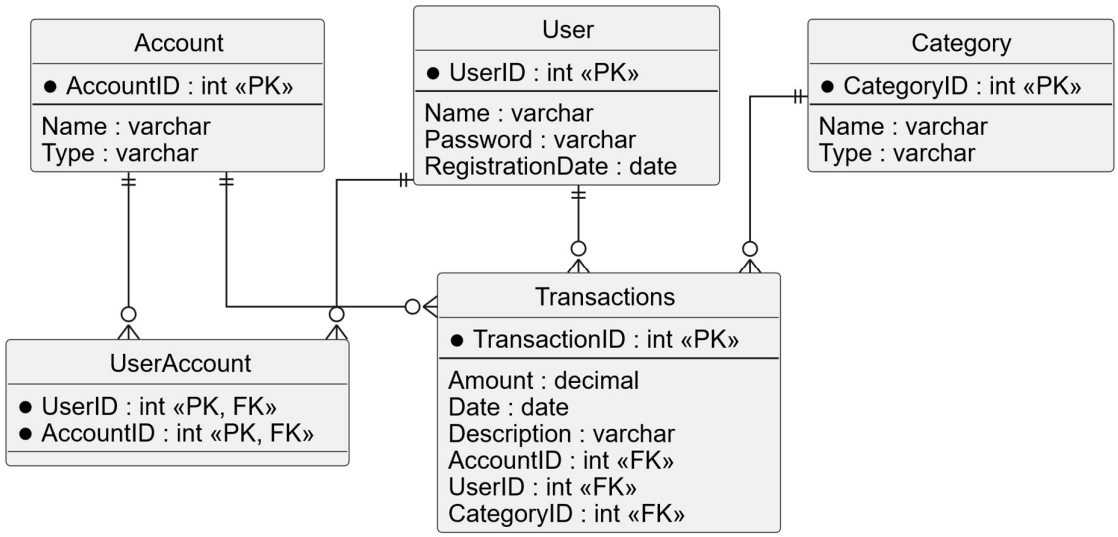
I have designed 5 tables in total:

1. **User** – contains information about users such as name, hashed password and registration data
2. **Account** - contains information about accounts such as account name and type
3. **UserAccount** – is a linking table for Users and Account. It is needed because Account can be assigned to many users
4. **Category** – contains information about available categories such as name and type(income/expense)

[15] https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/

5. **Transaction** – contains information about all transactions such as amount, creation date, description, AccountID, UserID and CategoryID

I have created UML diagram via draw.io that's shows tables and their entity relations:



Parameterized SQL queries

I knew that I am going to have many sql queries with different parameters, so I decided to learn how "sqlite-jdbc"[16] library that I use in my project implement paremeterized queries. After completing a little research I learn that:

1. This library contains PreparedStatement interface which let you write SQL template using "?" as a placeholder for actual values.
2. To do that, you have to use method prepareStatement() of interface Connection which accepts SQL template as an argument.
   For example:

```
String sql = "INSERT INTO Transactions (Amount, Date,
Description, UserID) VALUES (?, ?, ?, ?)";

Connection conn = getConnection();
PreparedStatement pstmt = conn.prepareStatement(sql));
```

3. After that you must bind actual value to place holder by using stricktly typed methods such as setDouble, setString, setInt and so on.
   For example:

[16] https://www.sqlitetutorial.net/sqlite-java/
https://www.geeksforgeeks.org/java/how-to-use-preparedstatement-in-java/

```
pstmt.setDouble(1, 45.50);
pstmt.setString(2, "2026-03-24");
pstmt.setString(3, "Dinner at McDonald's");
pstmt.setInt(4, 1);
```

4. Finally, you have to execute by using **pstmt.executeQuery()** for SELECT and by using **pstmt.executeUpdate()** for INSERT, UPDATE and DELETE.

A side effect of using parameterized SQL queries is that it prevents SQL Injection by treating parameters as literal values. So user will not be able to write piece of SQL query instead.

Retrieving data from database

When we apply **pstmt.executeQuery()** for parametrized query, it returns **ResultSet** object. It acts as a cursor which goes through all rows. In order to go to next row, we need to use method **next().** If there is no next raw, it returns "false". To get specific fields in row, we need to use strictly typed getters such as getString, getDouble and so on. It accepts name of column as an argument.

Example:

```
List<Transaction> transactionList = new ArrayList<>();
while (rs.next()) {

    int id = rs.getInt("TransactionID");
    double amount = rs.getDouble("Amount");
    String date = rs.getString("Date");
    String description = rs.getString("Description");

    transactionList.add(new Transaction
                          (id, amount, date, description, ...));

}
```

Also, I would like to mention **Commint modes** in JDBC library. Basically, JDBC library supports auto-commit mode and manual mode that can be swithced by using **conn.setAutoCommit(false/true).**

Auto-commit mode(default mode) instantly saves result of execution of eacj SQL statement. When it is turned off, commands are executed in memory, but stored temporary area and doesn't save to actual database. In order to save results of all commands, you need to call **conn.commit().** In case of error, it is possible to cancel all commands by using the command **conn.rollback()** which leaves database as it was before.

I decided to mention it because I am going to use it a lot when I will update or delete data in several tables.

I am going to have different SQL queries related to Dashboard, Categories, Accounts, User and Transaction. Now, I am going to list all queries.

Dashboard:

Total Monthly Income/Expense will be seen on the dashboard. For that purpose, I wrote query that will count sum of amounts in needed date range(current month):

```
"SELECT SUM(t.Amount) FROM Transactions t " +
    "JOIN Category c ON t.CategoryID = c.CategoryID " +
    "WHERE c.Type = ? AND t.CreationDate >= ? AND t.CreationDate < ?"
```

Categories:

For categories, I am going to have several queries for selecting all categories, creating new categories, updating categories and deleting categories.

Selecting	`"SELECT CategoryID, Name, Type FROM Category ORDER BY Type, Name"`
Creating	`"INSERT INTO Category (Name, Type) VALUES (?, ?)"`
Updating	`"UPDATE Category SET Name = ?, Type = ? WHERE CategoryID = ?"`

Deleting	`"DELETE FROM Transactions WHERE CategoryID = ?"`
	&
	`"DELETE FROM Category WHERE CategoryID = ?"`

Accounts:

For getting all accounts with their users, I wrote such a query:

```
"SELECT a.AccountID, a.Type, a.Name, COALESCE(GROUP_CONCAT(DISTINCT u.Name), '') AS
Users " +
    "FROM Account a " +
    "LEFT JOIN UserAccount ua ON ua.AccountID = a.AccountID " +
    "LEFT JOIN User u ON u.UserID = ua.UserID " +
    "GROUP BY a.AccountID, a.Type, a.Name " +
    "ORDER BY a.Name"
```

It selects AccountID, Type and Name. **GROUP_CONCAT(DISTINCT u.Name)** selects all distinct user names and group them into one string with comma separetor. The key word **COALESCE** returns Null value if there is no user attached to account. First **LEFT JOIN** connects Account to UserAccount and takes all UserAccount record with current account. Second **LEFT JOIN** connects UserAccount to User table and take all Users that are in needed UserAccount row. The feature of **LEFT JOIN** is that it keeps all account, even if there is no user assigned to this account in UserAccount table. **GROUP BY** looks for the same values in columns AccountID, Type & Name and as a result group them into one row. **ORDER BY** simply sorts row in alphabetic order according to names.

For creating the account and assigning all user, there are two queries:

```
"INSERT INTO Account (Name, Type) VALUES (?, ?)"
"INSERT INTO UserAccount (UserID, AccountID) VALUES (?, ?)"
```

For getting user IDs:

```
"SELECT UserID FROM UserAccount WHERE AccountID = ?"
```

For updating Account with users, we need to update Account table, delete all records from UserAccount and Reinsert new records into UserAccount table. We need to do that when we assigned new users. In order to make it easy, we just deleted all corresponding records and reinserted updated information.

```
"UPDATE Account SET Name = ?, Type = ? WHERE AccountID = ?"
"DELETE FROM UserAccount WHERE AccountID = ?"
"INSERT INTO UserAccount (UserID, AccountID) VALUES (?, ?)"
```

For deleting account, we need to delete all record with this account in Transactions table, Account table, and UserAccount table.

```
"DELETE FROM Transactions WHERE AccountID = ?"
"DELETE FROM UserAccount WHERE AccountID = ?"
"DELETE FROM Account WHERE AccountID = ?"
```

Users:

For getting all users:

```
"SELECT UserID, Name, Password, RegistrationDate FROM User ORDER BY Name"
```

For updating user information:

```
"UPDATE User SET Name = ? WHERE UserID = ?"
```

For creating a new user:

```
"INSERT INTO User (Name, Password, RegistrationDate) VALUES (?, ?, ?)"
```

To delete users from all tables, we have to do several steps:

1. Delete all records with needed user from Transactions table:
   ```
   "DELETE FROM Transactions WHERE UserID = ?"
   ```
2. Delete all records with needed user from UserAccount table:
   ```
   "DELETE FROM UserAccount WHERE UserID = ?"
   ```

3. Some accounts might be linked to only one user. If we delete this user, we have to delete account as well. To do that, we find all accounts that are assigned to user that we want to delete(We need to execute this query before steps 1,2):
   ```
   "SELECT AccountID FROM UserAccount WHERE UserID = ?"
   ```
   After that, we count how many records account has in UserAccount table. If it count = 0, it means that this account is not assigned to any user and it must be deleted by queries:
   ```
   "DELETE FROM Transactions WHERE AccountID = ?"
   "DELETE FROM Account WHERE AccountID = ?"
   ```
4. Finally, we can delete user from User table:
   ```
   "DELETE FROM User WHERE UserID = ?"
   ```

Transactions:

For getting all transactions with parameter to control number of transactions:

```
"SELECT t.TransactionID, c.Type, c.Name AS CategoryName, t.Amount, " +
"t.CreationDate, u.Name AS UserName, a.Name AS AccountName, t.Description " +
"FROM Transactions t " +
"JOIN Category c ON t.CategoryID = c.CategoryID " +
"JOIN User u ON t.UserID = u.UserID " +
"JOIN Account a ON t.AccountID = a.AccountID " +
"ORDER BY t.CreationDate DESC, t.TransactionID DESC " +
"LIMIT ?";
```

Inner joins are used to link IDs from transactions table to Categories, Users and Accounts. It sorted in such a way, so newest transactions appears first.

For searching and retrieving transactions based on specific filters, I will add parameters to base SQL query according to filters that are activated by user.

The base sql query:

```
"SELECT t.TransactionID, c.Type, c.Name AS CategoryName, t.Amount, " +
"t.CreationDate, u.Name AS UserName, a.Name AS AccountName, t.Description " +
"FROM Transactions t " +
"JOIN Category c ON t.CategoryID = c.CategoryID " +
"JOIN User u ON t.UserID = u.UserID " +
"JOIN Account a ON t.AccountID = a.AccountID WHERE 1=1 "
```

Because I need to add parameters to this query, I used a trick with **WHERE 1=1** to get rid of checking if it is a first condition added. It works because 1=1 is always true and doesn't affect the query. It is a common trick used in dynamic SQL queries generation[17].

I will add such part to base sql query:

SearchText	`" AND (t.Description LIKE ? OR c.Name LIKE ? OR u.Name LIKE ?)"`

[17] https://pushmetrics.io/blog/why-use-where-1-1-in-sql-queries-exploring-the-surprising-benefits-of-a-seemingly-redundant-clause/

Type	" AND c.Type = ?"
User	" AND u.Name = ?"
Account	" AND a.Name = ?"
Catergory	" AND c.Name = ?"
Date from	" AND t.CreationDate >= ?"
Date to	" AND t.CreationDate <= ?"
Min amount	" AND t.Amount >= ?"
Max amount	" AND t.Amount <= ?"

For getting all available accounts for a specific user:

```
"SELECT a.Name " +
"FROM Account a " +
"JOIN UserAccount ua ON ua.AccountID = a.AccountID " +
"WHERE ua.UserID = ? " +
"ORDER BY a.Name";
```

For getting categories by type:

```
"SELECT Name FROM Category WHERE Type = ? ORDER BY Name"
```

In order to add new transaction, we will need 3 separate queries. Before adding tranasction, we have to retrieve the accountID and categoryID.

Finding accountID based on entered data:

```
"SELECT a.AccountID " +
"FROM Account a " +
"JOIN UserAccount ua ON ua.AccountID = a.AccountID " +
"WHERE a.Name = ? AND ua.UserID = ?";
```

Finding categoryID based on entered data:

```
"SELECT CategoryID FROM Category WHERE Name = ? AND Type = ?"
```

After receiving categoryID and account ID, we can use these data for insetring query:

```
"INSERT INTO Transactions (Amount, CreationDate, Description, AccountID, UserID, CategoryID)
VALUES (?, ?, ?, ?, ?, ?)"
```

In my program user will be able to change amount and description of existing transaction. To do that, I wrote this query:

```
"UPDATE Transactions SET Amount = ?, Description = ? WHERE TransactionID = ?"
```

In order to delete transaction, there is such a query:

```
"DELETE FROM Transactions WHERE TransactionID = ?"
```

Algorithms

To secure users date, I created PasswordHashUtil class that contains hash function[18]. Hash function is a mathematical algorithm that takes an input of any size and converts it into a fixed-length string of characters. This output is called a "hash" or "digest". In my program, hash function is applied to passwords before they are insert into database. Also, it is applied when user tries to log-in. It hashes the password user entered and compares it with information from database. As a base for my hash function, I used the standard implementation of built-in method hashcode() in java[19].

Here is the code for my hash function:

```
public static String hash(String value) {
    if (value == null) {
        throw new IllegalArgumentException("Password cannot be null");
    }

    long hash = 17;
    for (int i = 0; i < value.length(); i++) {
        hash = hash * 31 + value.charAt(i);
    }

    return String.format("%08x", hash & 0xffffffffL);
}
```

Steps:

1. Checking that argument is not an empty string
2. Declaring variable hash with initial value of 17
3. Then, we go through all letters in string and every time multiply variable hash by 31 and add numerical value of current letter.
4. Then. we use String.format() method to convert integer value of hash into 8-character hexadecimal string.

Notes:

[18] https://www.geeksforgeeks.org/dsa/hash-functions-and-list-types-of-hash-functions/
[19] https://www.baeldung.com/java-hashcode#a-naive-hashcode-implementation

Initial hash number and multiplier that is used in the loop should be prime numbers to get rid of possible collision[20].

After completing the loop, the resulting hash can become a very large number. Therefore, I used a bitwise AND operator to cut the value, ensuring it only keeps the lowest 32 bits.[21]

Technical Solution

Below, I am going to attach the code of all Java classes and layout files that I have in my application. Also, to explain the code, I added comments into the code.

[20] https://www.quora.com/Why-is-it-recommended-to-use-a-prime-number-in-hash-code-calculation#:~:text=Prime%20numbers%20are%20typically%20used,multiple%20of%20that%20common%20factor.&text=A%20hash%20table%20and%20a,the%20address%20in%20the%20array.
[21] https://stackoverflow.com/questions/12772939/java-storing-two-ints-in-a-long

Moreover, I attached all fxml layouts that I use in my application. As I already mentioned, I used visual layout tool SceneBuilder to create them. Due to that reason, I will not add comments to layouts.
Finally, I wrote simple styles.css file with styles for UI layouts. I imported them in code:

```
scene.getStylesheets().add(String.valueOf(MainWindow.class.getResource("style.css")));
```

In my technical solution there are some things that I will use in various parts of the project. I will list them below to make it easier to understand my code:

1. When I load data in table, I will use ObservableList[22] instead of ArrayList ObservableList is an analogue of ArrayList that is required for the proper work of JavaFX tables
2. In controller classes, all variables injected from UI components will be notated with @**FXML** before variable
3. In DAO, I made all methods static, so they can be accessed without need to create an instance of DAO class

User.java

```
package timur.popov.financemanager.model;

import java.util.List;

/**
 * Basic user class used in Model and Controller
 */
public class User {
    private final int userId;
    private final String name;
    private final String registrationDate;
    private final String hashedPassword;

    public User(int userId, String name, String registrationDate, String hashedPassword, List<Account> ignoredAccountList) {
        this.userId = userId;
        this.name = name;
        this.registrationDate = registrationDate;
        this.hashedPassword = hashedPassword;
    }

    public int getUserId() {
        return userId;
    }

    public String getName() {
        return name;
    }

    public String getRegistrationDate() {
        return registrationDate;
    }

    public String getPassword() {
        return hashedPassword;
    }
}
```

[22] https://stackoverflow.com/questions/41920217/what-is-the-difference-between-arraylist-and-observablelist

Transaction.java

```java
package timur.popov.financemanager.model;

/**
 * Transaction class with all corresponding fields.
 * Also, fields in this class are bound to columns in transaction table
 */
public class Transaction {
    private int id;
    private String type;
    private String category;
    private double amount;
    private String date;
    private String user;
    private String account;
    private String description;

    /**
     * Used for transactions that are going to be inserted into database
     * Id doesn't need because, it will auto increment in database
     */
    public Transaction(String type, String category, double amount, String date, String user, String account, String description) {
        this.id = -1;
        this.type = type;
        this.category = category;
        this.amount = amount;
        this.date = date;
        this.user = user;
        this.account = account;
        this.description = description;
    }
    //Used for categories that are loaded from database
    public Transaction(int id, String type, String category, double amount, String date, String user, String account, String description) {
        this.id = id;
        this.type = type;
        this.category = category;
        this.amount = amount;
        this.date = date;
        this.user = user;
        this.account = account;
        this.description = description;
    }


    public String getType() { return type; }
    public String getCategory() { return category; }
    public double getAmount() { return amount; }
    public String getDate() { return date; }
    public String getUser() { return user; }
    public String getAccount() { return account; }
    public String getDescription() { return description; }
    public int getId() { return id; }


    public void setType(String type) { this.type = type; }
    public void setCategory(String category) { this.category = category; }
    public void setAmount(double amount) { this.amount = amount; }
    public void setDate(String date) { this.date = date; }
    public void setUser(String user) { this.user = user; }
    public void setAccount(String account) { this.account = account; }
    public void setDescription(String description) { this.description = description; }
    public void setId(int id) { this.id = id; }
}
```

Category.java

```java
package timur.popov.financemanager.model;

/**
 * Category class is used for keeping information about specific category
 */
public class Category {
    private int categoryId;
    private String type;
    private String name;

    /**
     * Used for categories that are going to be inserted into database
     * Id doesn't need because, it will auto increment in database
     */
    public Category(String type, String name) {
        this.categoryId = -1;
        this.type = type;
        this.name = name;
    }
    //Used for categories that are loaded from database
    public Category(int categoryId, String type, String name) {
        this.categoryId = categoryId;
        this.type = type;
        this.name = name;
    }


    public String getType() {
        return type;
    }

    public int getCategoryId() {
        return categoryId;
    }

    public String getName() {
        return name;
    }


    public void setType(String type) { this.type = type; }
    public void setName(String name) { this.name = name; }
    public void setCategoryId(int categoryId) { this.categoryId = categoryId; }
}
```

Account.java

```java
package timur.popov.financemanager.model;

/**
 * Account class keeps information about specific account.
 * users field stores information about all users through comma(e.g Alex, John, Ivan)
 */
public class Account {
    private final int ACCOUNT_ID;
    private String name;
    private String type;
    private String users;


    public Account(int ACCOUNT_ID, String name, String type, String users) {
        this.ACCOUNT_ID = ACCOUNT_ID;
        this.name = name;
        this.type = type;
        this.users = (users == null) ? "" : users;
    }

    public int getACCOUNT_ID() {
        return ACCOUNT_ID;
    }
```

```
    public String getName() {
        return name;
    }

    public String getType() {
        return type;
    }


    public String getUsers() {
        return users;
    }
}
```

Launcher.java

```
package timur.popov.financemanager;

import javafx.application.Application;

/**
 * Entry point of the program.
 */
public class Launcher {
    public static void main(String[] args) {
        Application.launch(MainWindow.class, args);
    }
}
```

MainWindow.java

```
package timur.popov.financemanager;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;
import timur.popov.financemanager.dao.DBUtil;

import java.io.IOException;

/**
 * Main JavaFX application shell in the MVC setup.
 * Initialize connection to database when program start anc close connection to db when program stops.
 * Creates a main window using main-window.fxml layout, add styles and set up basic parameters such as title and size.
 */
public class MainWindow extends Application {
    @Override
    public void init() throws Exception {
        super.init();
        new DBUtil().initializeDatabase();
    }

    @Override
    public void stop() throws Exception {
        DBUtil.closeConnection();
        super.stop();
    }

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource("view/main-window.fxml"));
        Scene scene = new Scene(fxmlLoader.load(), 1024, 768);
        scene.getStylesheets().add(String.valueOf(MainWindow.class.getResource("style.css")));
        stage.setTitle("Budget Tracker");
        stage.setScene(scene);
        stage.setResizable(false);
        stage.show();
```

```
    }
}
```

DialogUtil.java

```
package timur.popov.financemanager;

import javafx.scene.control.Alert;
import javafx.scene.control.ButtonType;
import javafx.scene.control.Dialog;

import java.util.Optional;

/**
 * Utility class that implement all dialog behavior to keep controllers small. Example of encapsulation.
 */
public final class DialogUtil {
    private DialogUtil() {
    }

    public static void showInfo(String title, String message) {
        showAlert(Alert.AlertType.INFORMATION, title, message);
    }

    public static void showError(String title, String message) {
        showAlert(Alert.AlertType.ERROR, title, message);
    }

    public static <T> Optional<T> showUntilPresent(Dialog<T> dialog) {
        Optional<T> result;
        /*
         * Repeats until controller returns a non-null result.
         * It's needed when user tries to create new transaction, category, account and so on.
         * The result variable contains an Optional instance with the data entered by the user.
         */
        do {
            result = dialog.showAndWait();
        } while (result.isEmpty());
        return result;
    }

    private static void showAlert(Alert.AlertType type, String title, String message) {
        Alert alert = new Alert(type, message, ButtonType.OK);
        alert.setTitle(title);
        alert.setHeaderText(null);
        alert.showAndWait();
    }
}
```

PasswordHashUtil.java

```
package timur.popov.financemanager;

/**
 * Class implements hash function for registering and logging
 * matches method compares hashed value from db and entered password when user try to log-in
 */
public final class PasswordHashUtil { private PasswordHashUtil() {
    }

    public static String hash(String value) {
        if (value == null) {
            throw new IllegalArgumentException("Password cannot be null");
        }

        long hash = 17;
        for (int i = 0; i < value.length(); i++) {
            hash = hash * 31 + value.charAt(i);
```

```
        }

        return String.format("%08x", hash & 0xffffffffL);
    }

    public static boolean matches(String rawPassword, String storedHash) {
        if (rawPassword == null || storedHash == null) {
            return false;
        }

        return hash(rawPassword).equals(storedHash);
    }
}
```

UserSession.java

```
package timur.popov.financemanager;

import timur.popov.financemanager.model.User;

/**
 * Keeps information about current user
 * Implements Singleton pattern
 * Used by controllers to define current user or to change current user
 */
public final class UserSession {
    private static final UserSession INSTANCE = new UserSession();

    private User currentUser;

    private UserSession() {}

    public static UserSession getInstance() {
        return INSTANCE;
    }

    public User getCurrentUser() {
        return currentUser;
    }

    public void setCurrentUser(User user) {
        currentUser = user;
    }

    public void clear() {
        currentUser = null;
    }
}
```

DBUtil.java

```
package timur.popov.financemanager.dao;

import timur.popov.financemanager.MainWindow;

import java.nio.file.Files;
import java.nio.file.Paths;
import java.sql.*;

/**
 * Implementation of methods that open and close connection to database
 *
 */
public class DBUtil {
    private static final String URL = "jdbc:sqlite:app.db";
    private static Connection connection;

    public static Connection getConnection() throws SQLException {
        // Lazy initialization keeps startup simple while still sharing one JDBC connection.
```

```
        if (connection == null || connection.isClosed()) {
            connection = DriverManager.getConnection(URL);
        }
        return connection;
    }
    // Creates file and tables If database file doesn't exist or doesn't have some tables
    public void initializeDatabase() {
        try (Connection conn = getConnection();
             Statement stmt = conn.createStatement()) {

            //Read the file with sql code that creates all tables
            String sql = new String(Files.readAllBytes(
                    Paths.get(MainWindow.class.getResource("sql/creating_tables.sql").toURI())
            ));

            stmt.executeUpdate(sql);
            System.out.println("Database initialized successfully!");

        } catch (Exception e) {
            System.err.println("Error during database initialization:");
            e.printStackTrace();
        }
    }

    public static void closeConnection() {
        try {
            if (connection != null && !connection.isClosed()) {
                connection.close();
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

DashboardDAO.java:

```
package timur.popov.financemanager.dao;

import java.sql.*;
import java.time.LocalDate;

import static timur.popov.financemanager.dao.DBUtil.getConnection;

/**
 * DAO needed for proper work of dashboard
 * Have one method that returns total sum for current month by category type
 */
public class DashboardDAO {
    public static double sumCurrentMonthByType(String categoryType) {
        // Sets interval for current month
        LocalDate startOfMonth = LocalDate.now().withDayOfMonth(1);
        LocalDate startOfNextMonth = startOfMonth.plusMonths(1);

        String sql = "SELECT SUM(t.Amount) FROM Transactions t " +
                "JOIN Category c ON t.CategoryID = c.CategoryID " +
                "WHERE c.Type = ? AND t.CreationDate >= ? AND t.CreationDate < ?";

        // Open connection and replace parameters(?) in query with variables
        try (Connection conn = getConnection();
             PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, categoryType);
            pstmt.setString(2, startOfMonth.toString());
            pstmt.setString(3, startOfNextMonth.toString());

            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    // return value in column 1 of result set. There is only one column in result set because SUM(t.Amount)
                    return rs.getDouble(1);
```

```
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    //Return 0 if no transaction in current month
    return 0.0;
  }
}
```

AccountsDAO.java

```
package timur.popov.financemanager.dao;

import timur.popov.financemanager.model.Account;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.List;

import static timur.popov.financemanager.dao.DBUtil.getConnection;

/**
 * Implements CRUD operation for account.
 */
public class AccountsDAO {

  public static List<Account> getAllAccountsWithUsers() {
    String sql = "SELECT a.AccountID, a.Type, a.Name, COALESCE(GROUP_CONCAT(DISTINCT u.Name), '') AS Users " +
          "FROM Account a " +
          "LEFT JOIN UserAccount ua ON ua.AccountID = a.AccountID " +
          "LEFT JOIN User u ON u.UserID = ua.UserID " +
          "GROUP BY a.AccountID, a.Type, a.Name " +
          "ORDER BY a.Name";

    List<Account> accounts = new ArrayList<>();

    try (Connection conn = getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql);
        ResultSet rs = pstmt.executeQuery()) {

      while (rs.next()) {
        accounts.add(new Account(
            rs.getInt("AccountID"),
            rs.getString("Name"),
            rs.getString("Type"),
            rs.getString("Users")
        ));
      }
    } catch (SQLException e) {
      e.printStackTrace();
    }

    return accounts;
  }

  public static boolean createAccountWithUsers(String type, String accountName, List<Integer> userIds) {
    if (type == null || accountName == null || accountName.isBlank() || userIds == null || userIds.isEmpty()) {
      return false;
    }

    String insertAccountSql = "INSERT INTO Account (Name, Type) VALUES (?, ?)";
    String insertUserAccountSql = "INSERT INTO UserAccount (UserID, AccountID) VALUES (?, ?)";

    Connection conn = null;
    try {
```

```
        conn = getConnection();
        /*
         * Turning off AutoCommit because we will execute several commands because
         * we don't want changes to be recorded into database until all command are executed without errors
         */
        conn.setAutoCommit(false);

        int accountId;
        //Statement.RETURN_GENERATED_KEYS allows us to get generated AccountID after inserting new account
        try (PreparedStatement accountStmt = conn.prepareStatement(insertAccountSql, Statement.RETURN_GENERATED_KEYS)) {
            accountStmt.setString(1, accountName);
            accountStmt.setString(2, type);
            accountStmt.executeUpdate();

            try (ResultSet generatedKeys = accountStmt.getGeneratedKeys()) {
                if (!generatedKeys.next()) {
                    // rollback allows us to undo all changes made to database
                    conn.rollback();
                    return false;
                }
                //Indexes starts from 1
                accountId = generatedKeys.getInt(1);
            }
        }

        try (PreparedStatement userAccountStmt = conn.prepareStatement(insertUserAccountSql)) {
            // Goes through all users and replace parameters in query with variable values and add them to batch
            for (Integer userId : userIds) {
                userAccountStmt.setInt(1, userId);
                userAccountStmt.setInt(2, accountId);
                userAccountStmt.addBatch();
            }
            // Basically execute several sql commands at once
            userAccountStmt.executeBatch();
        }
        //apply all changes to database
        conn.commit();
        return true;
    } catch (SQLException e) {
        if (conn != null) {
            try {
                conn.rollback();
            } catch (SQLException rollbackException) {
                rollbackException.printStackTrace();
            }
        }
        e.printStackTrace();
        return false;
    } finally {
        if (conn != null) {
            try {
                // Because there is only one instance of connection to database we have to return AutoCommit to true.
                conn.setAutoCommit(true);
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}

public static List<Integer> getUserIdsForAccount(int accountId) {
    List<Integer> userIds = new ArrayList<>();
        String sql = "SELECT UserID FROM UserAccount WHERE AccountID = ?";

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, accountId);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                userIds.add(rs.getInt("UserID"));
            }
```

```
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }


    return userIds;
}

public static boolean updateAccountWithUsers(int accountId, String type, String accountName, List<Integer> userIds) {
    if (accountName == null || accountName.isBlank() || type == null || type.isBlank() || userIds == null || userIds.isEmpty()) {
        return false;
    }

    String updateAccountSql = "UPDATE Account SET Name = ?, Type = ? WHERE AccountID = ?";
    String deleteMappingsSql = "DELETE FROM UserAccount WHERE AccountID = ?";
    String insertMappingSql = "INSERT INTO UserAccount (UserID, AccountID) VALUES (?, ?)";

    Connection conn = null;
    try {
        conn = getConnection();
        // Same idea as in createAccountWithUsers() method
        conn.setAutoCommit(false);

        try (PreparedStatement updateStmt = conn.prepareStatement(updateAccountSql)) {
            updateStmt.setString(1, accountName);
            updateStmt.setString(2, type);
            updateStmt.setInt(3, accountId);
            updateStmt.executeUpdate();
        }

        try (PreparedStatement deleteStmt = conn.prepareStatement(deleteMappingsSql)) {
            deleteStmt.setInt(1, accountId);
            deleteStmt.executeUpdate();
        }

        try (PreparedStatement insertStmt = conn.prepareStatement(insertMappingSql)) {
            for (Integer userId : userIds) {
                insertStmt.setInt(1, userId);
                insertStmt.setInt(2, accountId);
                insertStmt.addBatch();
            }
            insertStmt.executeBatch();
        }

        conn.commit();
        return true;
    } catch (SQLException e) {
        if (conn != null) {
            try {
                conn.rollback();
            } catch (SQLException rollbackException) {
                rollbackException.printStackTrace();
            }
        }
        e.printStackTrace();
        return false;
    } finally {
        if (conn != null) {
            try {
                conn.setAutoCommit(true);
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}

public static boolean deleteAccount(int accountId) {
    String deleteTransactionsSql = "DELETE FROM Transactions WHERE AccountID = ?";
    String deleteMappingsSql = "DELETE FROM UserAccount WHERE AccountID = ?";
```

```
        String deleteAccountSql = "DELETE FROM Account WHERE AccountID = ?";

        Connection conn = null;
        try {
            conn = getConnection();
            conn.setAutoCommit(false);

            try (PreparedStatement pstmt = conn.prepareStatement(deleteTransactionsSql)) {
                pstmt.setInt(1, accountId);
                pstmt.executeUpdate();
            }

            try (PreparedStatement pstmt = conn.prepareStatement(deleteMappingsSql)) {
                pstmt.setInt(1, accountId);
                pstmt.executeUpdate();
            }

            int affected;
            try (PreparedStatement pstmt = conn.prepareStatement(deleteAccountSql)) {
                pstmt.setInt(1, accountId);
                affected = pstmt.executeUpdate();
            }

            conn.commit();
            return affected > 0;
        } catch (SQLException e) {
            if (conn != null) {
                try {
                    conn.rollback();
                } catch (SQLException rollbackException) {
                    rollbackException.printStackTrace();
                }
            }
            e.printStackTrace();
            return false;
        } finally {
            if (conn != null) {
                try {
                    conn.setAutoCommit(true);
                } catch (SQLException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

CategoriesDAO.java

```
package timur.popov.financemanager.dao;

import timur.popov.financemanager.model.Category;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import static timur.popov.financemanager.dao.DBUtil.getConnection;

/**
 * DAO for category CRUD operations.
 */
public class CategoriesDAO {

    public static List<Category> getAllCategories() {
        List<Category> categories = new ArrayList<>();
        String sql = "SELECT CategoryID, Name, Type FROM Category ORDER BY Type, Name";
```

```
    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql);
         ResultSet rs = pstmt.executeQuery()) {
        //Fill out ArrayList with all available categories
        while (rs.next()) {
            categories.add(new Category(
                    rs.getInt("CategoryID"),
                    rs.getString("Type"),
                    rs.getString("Name")
            ));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }

    return categories;
}

public static boolean createCategory(String name, String type) {
    if (name == null || name.isBlank() || type == null || type.isBlank()) {
        return false;
    }

    String insertSql = "INSERT INTO Category (Name, Type) VALUES (?, ?)";

    try (Connection conn = getConnection();
         PreparedStatement insertStmt = conn.prepareStatement(insertSql)) {
        insertStmt.setString(1, name);
        insertStmt.setString(2, type);
        return insertStmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public static boolean updateCategory(int categoryId, String name, String type) {
    String sql = "UPDATE Category SET Name = ?, Type = ? WHERE CategoryID = ?";

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, name);
        pstmt.setString(2, type);
        pstmt.setInt(3, categoryId);
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public static boolean deleteCategory(int categoryId) {
    String deleteTransactionsSql = "DELETE FROM Transactions WHERE CategoryID = ?";
    String deleteCategorySql = "DELETE FROM Category WHERE CategoryID = ?";

    Connection conn = null;
    try {
        conn = getConnection();
        /*
         * Turning off AutoCommit because we will execute several commands because
         * we don't want changes to be recorded into database until all command are executed without errors
         */
        conn.setAutoCommit(false);

        try (PreparedStatement deleteTransactions = conn.prepareStatement(deleteTransactionsSql)) {
            deleteTransactions.setInt(1, categoryId);
            deleteTransactions.executeUpdate();
        }

        int affected;
```

```
        try (PreparedStatement deleteCategory = conn.prepareStatement(deleteCategorySql)) {
            deleteCategory.setInt(1, categoryId);
            affected = deleteCategory.executeUpdate();
        }

        conn.commit();
        return affected > 0;
    } catch (SQLException e) {
        if (conn != null) {
            try {
                conn.rollback();
            } catch (SQLException rollbackException) {
                rollbackException.printStackTrace();
            }
        }
        e.printStackTrace();
        return false;
    } finally {
        if (conn != null) {
            try {
                conn.setAutoCommit(true);
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
  }
}
```

UserDAO.java

```
package timur.popov.financemanager.dao;

import timur.popov.financemanager.model.User;

import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

import static timur.popov.financemanager.dao.DBUtil.getConnection;

/**
 * DAO that handles CRUD operation for user.
 */
public class UserDAO {

  public static List<User> getAllUsers() {
    List<User> users = new ArrayList<>();
    String sql = "SELECT UserID, Name, Password, RegistrationDate FROM User ORDER BY Name";

    try (Connection conn = getConnection();
         Statement stmt = conn.createStatement();
         ResultSet rs = stmt.executeQuery(sql)) {
        // Retrieve all users from database
        while (rs.next()) {
            User user = new User(
                    rs.getInt("UserID"),
                    rs.getString("Name"),
                    rs.getString("RegistrationDate"),
                    rs.getString("Password"),
                    null
            );
            users.add(user);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return users;
  }
```

```
public static boolean updateUserName(int userId, String newName) {
    String sql = "UPDATE User SET Name = ? WHERE UserID = ?";

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, newName);
        pstmt.setInt(2, userId);
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public static boolean createUser(String name, String hashedPassword) {
    // We don't need to turn off AutoCommit because there is only one query that will change database
    String nextIdSql = "SELECT COALESCE(MAX(UserID), 0) + 1 FROM User";
    String insertSql = "INSERT INTO User (UserID, Name, Password, RegistrationDate) VALUES (?, ?, ?, ?)";

    try (Connection conn = getConnection();
         Statement idStmt = conn.createStatement();
         ResultSet rs = idStmt.executeQuery(nextIdSql)) {

        int nextId = 1;
        if (rs.next()) {
            nextId = rs.getInt(1);
        }

        try (PreparedStatement pstmt = conn.prepareStatement(insertSql)) {
            pstmt.setInt(1, nextId);
            pstmt.setString(2, name);
            pstmt.setString(3, hashedPassword);
            pstmt.setString(4, LocalDate.now().toString());
            return pstmt.executeUpdate() > 0;
        }
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public static boolean deleteUser(int userId) {
    Connection conn = null;

    try {
        conn = getConnection();
        /*
         * Turning off AutoCommit because we will execute several commands because
         * we don't want changes to be recorded into database until all command are executed without errors
         */
        conn.setAutoCommit(false);

        List<Integer> linkedAccountIds = new ArrayList<>();
        try (PreparedStatement selectAccounts = conn.prepareStatement(
                "SELECT AccountID FROM UserAccount WHERE UserID = ?")) {
            selectAccounts.setInt(1, userId);
            try (ResultSet rs = selectAccounts.executeQuery()) {
                while (rs.next()) {
                    //getting all accountsID with user that we want to delete
                    linkedAccountIds.add(rs.getInt("AccountID"));
                }
            }
        }

        try (PreparedStatement deleteUserTransactions = conn.prepareStatement(
                "DELETE FROM Transactions WHERE UserID = ?")) {
            deleteUserTransactions.setInt(1, userId);
            deleteUserTransactions.executeUpdate();
        }

        try (PreparedStatement deleteUserMappings = conn.prepareStatement(
```

```
            "DELETE FROM UserAccount WHERE UserID = ?")) {
        deleteUserMappings.setInt(1, userId);
        deleteUserMappings.executeUpdate();
      }

      try (PreparedStatement countMappings = conn.prepareStatement(
          "SELECT COUNT(*) FROM UserAccount WHERE AccountID = ?");
       PreparedStatement deleteAccountTransactions = conn.prepareStatement(
           "DELETE FROM Transactions WHERE AccountID = ?");
       PreparedStatement deleteAccount = conn.prepareStatement(
           "DELETE FROM Account WHERE AccountID = ?")) {

        for (Integer accountId : linkedAccountIds) {
          countMappings.setInt(1, accountId);
          try (ResultSet rs = countMappings.executeQuery()) {
            //checking if there is any records with accounts from linkedAccountIds
            if (rs.next() && rs.getInt(1) == 0) {
              //If no, we delete those accounts because they have no linked user anymore
              deleteAccountTransactions.setInt(1, accountId);
              deleteAccountTransactions.executeUpdate();

              deleteAccount.setInt(1, accountId);
              deleteAccount.executeUpdate();
            }
          }
        }
      }

      try (PreparedStatement deleteUser = conn.prepareStatement(
          "DELETE FROM User WHERE UserID = ?")) {
        deleteUser.setInt(1, userId);
        boolean deleted = deleteUser.executeUpdate() > 0;

        conn.commit();
        return deleted;
      }
    } catch (SQLException e) {
      if (conn != null) {
        try {
          conn.rollback();
        } catch (SQLException rollbackException) {
          rollbackException.printStackTrace();
        }
      }
      e.printStackTrace();
      return false;
    } finally {
      if (conn != null) {
        try {
          conn.setAutoCommit(true);
        } catch (SQLException e) {
          e.printStackTrace();
        }
      }
    }
  }
}
```

TransactionDAO.java

```
package timur.popov.financemanager.dao;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import timur.popov.financemanager.model.Transaction;

import java.sql.*;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneId;
```

```
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.List;

import static timur.popov.financemanager.dao.DBUtil.getConnection;

/**
 * DAO for transaction.
 * The whole filtering is here to keep MVC model
 */
public class TransactionsDAO {
   public static ObservableList<Transaction> getTransactions(int limit) {
         ObservableList<Transaction> transactionList = FXCollections.observableArrayList();

      String sql = "SELECT t.TransactionID, c.Type, c.Name AS CategoryName, t.Amount, " +
            "t.CreationDate, u.Name AS UserName, a.Name AS AccountName, t.Description " +
            "FROM Transactions t " +
            "JOIN Category c ON t.CategoryID = c.CategoryID " +
            "JOIN User u ON t.UserID = u.UserID " +
            "JOIN Account a ON t.AccountID = a.AccountID " +
            "ORDER BY t.CreationDate DESC, t.TransactionID DESC " +
            "LIMIT ?";

      try (Connection conn = getConnection();
           PreparedStatement pstmt = conn.prepareStatement(sql)) {

         pstmt.setInt(1, limit);

         try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
               Transaction trans = new Transaction(
                     rs.getInt("TransactionID"),
                     rs.getString("Type"),
                     rs.getString("CategoryName"),
                     rs.getDouble("Amount"),
                     rs.getString("CreationDate"),
                     rs.getString("UserName"),
                     rs.getString("AccountName"),
                     rs.getString("Description")
               );
               transactionList.add(trans);
            }
         }
      } catch (SQLException e) {
         e.printStackTrace();
      }
      return transactionList;
   }

   public static List<Transaction> searchTransactions(String searchText, String type, String user, String account, String category, LocalDate
from, LocalDate to, Double min, Double max) {
      List<Transaction> list = new ArrayList<>();
      // Base query that will be completed with filters
      StringBuilder sql = new StringBuilder(
            "SELECT t.TransactionID, c.Type, c.Name AS CategoryName, t.Amount, " +
                  "t.CreationDate, u.Name AS UserName, a.Name AS AccountName, t.Description " +
                  "FROM Transactions t " +
                  "JOIN Category c ON t.CategoryID = c.CategoryID " +
                  "JOIN User u ON t.UserID = u.UserID " +
                  "JOIN Account a ON t.AccountID = a.AccountID WHERE 1=1 "
      );
      //Using Object type because parameters have different types
      List<Object> params = new ArrayList<>();

      //Add filters to query and save them to params list
      if (searchText != null && !searchText.isEmpty()) {
         sql.append(" AND (t.Description LIKE ? OR c.Name LIKE ? OR u.Name LIKE ?)");
         String likeValue = "%" + searchText + "%"; // % is a wildcard for “any sequence of characters”
         params.add(likeValue);
         params.add(likeValue);
         params.add(likeValue);
```

```
    }
    if (type != null) {
        sql.append(" AND c.Type = ?");
        params.add(type);
    }
    if (user != null) {
        sql.append(" AND u.Name = ?");
        params.add(user);
    }
    if (account != null) {
        sql.append(" AND a.Name = ?");
        params.add(account);
    }
    if (category != null) {
        sql.append(" AND c.Name = ?");
        params.add(category);
    }
    if (from != null) {
        sql.append(" AND t.CreationDate >= ?");
        params.add(from.toString());
    }
    if (to != null) {
        sql.append(" AND t.CreationDate <= ?");
        params.add(to.toString());
    }
    if (min != null) {
        sql.append(" AND t.Amount >= ?");
        params.add(min);
    }
    if (max != null) {
        sql.append(" AND t.Amount <= ?");
        params.add(max);
    }

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql.toString())) {
        // Parameters list save the order of added parameters.
        for (int i = 0; i < params.size(); i++) {
            // i + 1 because in list indexes starts from 0, but in pstmt it starts from 1.
            pstmt.setObject(i + 1, params.get(i));
        }
        ResultSet rs = pstmt.executeQuery();
        while (rs.next()) {
            Transaction trans = new Transaction(
                    rs.getInt("TransactionID"),
                    rs.getString("Type"),
                    rs.getString("CategoryName"),
                    rs.getDouble("Amount"),
                    rs.getString("CreationDate"),
                    rs.getString("UserName"),
                    rs.getString("AccountName"),
                    rs.getString("Description")
            );
            list.add(trans);
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return list;
}

// Used in TransactionController when filter options are loaded(e.g. user names, category names and account names)
public static List<String> getColumnData(String tableName, String columnName) {
    List<String> options = new ArrayList<>();
    String sql = "SELECT " + columnName + " FROM " + tableName;

    try (Connection conn = getConnection();
         Statement stmt = conn.createStatement();
         ResultSet rs = stmt.executeQuery(sql)) {

        while (rs.next()) {
```

```
            options.add(rs.getString(columnName));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return options;
}
// Used in QuickActionController when user create new transaction and need to choose account
public static List<String> getAccountsForUser(int userId) {
    List<String> accounts = new ArrayList<>();
    String sql = "SELECT a.Name " +
            "FROM Account a " +
            "JOIN UserAccount ua ON ua.AccountID = a.AccountID " +
            "WHERE ua.UserID = ? " +
            "ORDER BY a.Name";

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, userId);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                accounts.add(rs.getString("Name"));
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }

    return accounts;
}
// Used in QuickActionController when user create new transaction and need to choose category
public static List<String> getCategoriesByType(String type) {
    List<String> categories = new ArrayList<>();
    String sql = "SELECT Name FROM Category WHERE Type = ? ORDER BY Name";

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setString(1, type);
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                categories.add(rs.getString("Name"));
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }

    return categories;
}

public static boolean createTransaction(int userId, String accountName, String categoryName, String type, double amount, String
description) {
    String accountIdSql = "SELECT a.AccountID " +
            "FROM Account a " +
            "JOIN UserAccount ua ON ua.AccountID = a.AccountID " +
            "WHERE a.Name = ? AND ua.UserID = ?";
    String categoryIdSql = "SELECT CategoryID FROM Category WHERE Name = ? AND Type = ?";
    String insertSql = "INSERT INTO Transactions (Amount, CreationDate, Description, AccountID, UserID, CategoryID) VALUES (?, ?, ?, ?, ?,
?)";

    try (Connection conn = getConnection()) {
        Integer accountId = null;
        try (PreparedStatement accountStmt = conn.prepareStatement(accountIdSql)) {
            accountStmt.setString(1, accountName);
            accountStmt.setInt(2, userId);
            try (ResultSet accountRs = accountStmt.executeQuery()) {
                if (accountRs.next()) {
                    accountId = accountRs.getInt("AccountID");
                }
            }
        }
```

```
        Integer categoryId = null;
        try (PreparedStatement categoryStmt = conn.prepareStatement(categoryIdSql)) {
            categoryStmt.setString(1, categoryName);
            categoryStmt.setString(2, type);
            try (ResultSet categoryRs = categoryStmt.executeQuery()) {
                if (categoryRs.next()) {
                    categoryId = categoryRs.getInt("CategoryID");
                }
            }
        }

        if (accountId == null || categoryId == null) {
            return false;
        }

        try (PreparedStatement insertStmt = conn.prepareStatement(insertSql)) {
            insertStmt.setDouble(1, amount);

            insertStmt.setString(2, LocalDate.now().toString());
            insertStmt.setString(3, description);
            insertStmt.setInt(4, accountId);
            insertStmt.setInt(5, userId);
            insertStmt.setInt(6, categoryId);
            return insertStmt.executeUpdate() > 0;
        }
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public static boolean updateTransactionBasic(int transactionId, double amount, String description) {
    String sql = "UPDATE Transactions SET Amount = ?, Description = ? WHERE TransactionID = ?";

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setDouble(1, amount);
        pstmt.setString(2, description);
        pstmt.setInt(3, transactionId);
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}

public static boolean deleteTransaction(int transactionId) {
    String sql = "DELETE FROM Transactions WHERE TransactionID = ?";

    try (Connection conn = getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {
        pstmt.setInt(1, transactionId);
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
        return false;
    }
}
}
```

DashboardController.java

```
package timur.popov.financemanager.controller.tabs;

import javafx.fxml.FXML;
import javafx.scene.control.*;
import timur.popov.financemanager.dao.DashboardDAO;
import timur.popov.financemanager.dao.TransactionsDAO;
import timur.popov.financemanager.model.Transaction;
```

```
/**
 * Controller for dashboard.fxml layout
 * Load total sum for current month by type
 * Load table with recent transactions
 */
public class DashboardController {
    @FXML public Label totalIncomeLabel;
    @FXML public Label totalExpenseLabel;
    @FXML public TableView<Transaction> recentTransactionsTable;

    @FXML
    public void initialize() {
        loadDashboardData();

    }

    private void loadDashboardData() {
        double income = DashboardDAO.sumCurrentMonthByType("Income");
        double expense = DashboardDAO.sumCurrentMonthByType("Expense");
        totalIncomeLabel.setText(String.valueOf(income));
        totalExpenseLabel.setText(String.valueOf(expense));

        recentTransactionsTable.setItems(TransactionsDAO.getTransactions(16));
    }
}
```

AccountsController.java

```
package timur.popov.financemanager.controller.tabs;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import timur.popov.financemanager.MainWindow;
import timur.popov.financemanager.controller.dialogs.CreatingAccountController;
import timur.popov.financemanager.dao.AccountsDAO;
import timur.popov.financemanager.model.Account;
import timur.popov.financemanager.DialogUtil;

import java.io.IOException;
import java.util.List;

/**
 * Controller for account management.
 */
public class AccountsController {
    @FXML private TextField searchAccountField;
    @FXML private Button searchButton;
    @FXML private Button addNewAccountButton;

    @FXML private TableView<Account> accountTableView;
    @FXML private TableColumn<Account, String> accountTypeColumn;
    @FXML private TableColumn<Account, String> accountNameColumn;
    @FXML private TableColumn<Account, String> accountUsersColumn;

    private final ObservableList<Account> accountData = FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        // Argument of new PropertyValueFactory<>() must have the same name as in Account class. Otherwise, they will not be bound and
there will be empty column
        accountTypeColumn.setCellValueFactory(new PropertyValueFactory<>("type"));
        accountNameColumn.setCellValueFactory(new PropertyValueFactory<>("name"));
        accountUsersColumn.setCellValueFactory(new PropertyValueFactory<>("users"));

        loadAccounts();
```

```
    accountTableView.setItems(accountData);

    //By clicking twice on specific raw, showEditDeleteDialog appears
    accountTableView.setRowFactory(tv -> {
        TableRow<Account> row = new TableRow<>();
        row.setOnMouseClicked(event -> {
            if (event.getClickCount() == 2 && !row.isEmpty()) {
                showEditDeleteDialog(row.getItem());
            }
        });
        return row;
    });

    searchButton.setOnAction(event -> handleSearch());
    addNewAccountButton.setOnAction(event -> openCreateAccountDialog());
}

private void openCreateAccountDialog() {
    try {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource("view/dialogs/create-account-dialog.fxml"));
        Dialog<CreatingAccountController.AccountData> dialog = new Dialog<>();
        dialog.setDialogPane(fxmlLoader.load());
        dialog.setTitle("Create New Account");
        dialog.getDialogPane().getStylesheets().add(MainWindow.class.getResource("style.css").toExternalForm());

        //Each dialog has own controller which handle entered data
        CreatingAccountController controller = fxmlLoader.getController();

        dialog.setResultConverter(dialogButton -> {
            if (dialogButton == ButtonType.APPLY) {
                // method buildResult() creates and returns AccountData instance with entered data
                return controller.buildResult();
            } else if (dialogButton == ButtonType.CLOSE) {
                // return empty data when operation is canceled
                return new CreatingAccountController.AccountData(null, null, List.of());
            }
            return null;
        });
        // Retrieving AccountData from dialog
        CreatingAccountController.AccountData data = DialogUtil.showUntilPresent(dialog).get();

        //If dialog was closed before completing, type = null -> method will stop
        if (data.type() == null) {
            return;
        }

        boolean created = AccountsDAO.createAccountWithUsers(data.type(), data.accountName(), data.userIds());
        if (created) {
            refreshAccountsView();
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

private void showEditDeleteDialog(Account account) {
    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
    alert.setTitle("Account Actions");
    alert.setHeaderText("Edit or Delete Account");
    alert.setContentText("Choose an action for: " + account.getName());

    ButtonType editButton = new ButtonType("Edit");
    ButtonType deleteButton = new ButtonType("Delete", ButtonBar.ButtonData.CANCEL_CLOSE);
    ButtonType cancelButton = new ButtonType("Cancel", ButtonBar.ButtonData.NO);
    alert.getButtonTypes().setAll(editButton, deleteButton, cancelButton);

    alert.showAndWait().ifPresent(type -> {
        if (type == editButton) {
            handleEdit(account);
        } else if (type == deleteButton) {
            handleDelete(account);
```

```
        }
    });
}

private void handleEdit(Account account) {
    try {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource("view/dialogs/create-account-dialog.fxml"));
        Dialog<CreatingAccountController.AccountData> dialog = new Dialog<>();
        dialog.setDialogPane(fxmlLoader.load());
        dialog.setTitle("Edit Account");
        dialog.getDialogPane().getStylesheets().add(MainWindow.class.getResource("style.css").toExternalForm());

        CreatingAccountController controller = fxmlLoader.getController();
        controller.setInitialData(account.getType(), account.getName(), AccountsDAO.getUserIdsForAccount(account.getACCOUNT_ID()));

        // Opening dialog for editing account data
        dialog.setResultConverter(dialogButton -> {
            if (dialogButton == ButtonType.APPLY) {
                return controller.buildResult();
            } else if (dialogButton == ButtonType.CLOSE) {
                return new CreatingAccountController.AccountData(null, null, List.of());
            }
            return null;
        });

        CreatingAccountController.AccountData data = DialogUtil.showUntilPresent(dialog).get();
        if (data.type() == null) {
            return;
        }
        boolean updated = AccountsDAO.updateAccountWithUsers(
                account.getACCOUNT_ID(),
                data.type(),
                data.accountName(),
                data.userIds()
        );

        if (updated) {
            refreshAccountsView();
        } else {
            DialogUtil.showError("Error", "Could not update account.");
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

private void handleDelete(Account account) {
    Alert confirm = new Alert(Alert.AlertType.CONFIRMATION);
    confirm.setTitle("Delete Account");
    confirm.setHeaderText("Delete selected account?");
    confirm.setContentText(account.getName());

    confirm.showAndWait().ifPresent(buttonType -> {
        if (buttonType != ButtonType.OK) {
            return;
        }

        boolean deleted = AccountsDAO.deleteAccount(account.getACCOUNT_ID());
        if (deleted) {
            refreshAccountsView();
        } else {
            DialogUtil.showError("Error", "Could not delete account.");
        }
    });
}

private void loadAccounts() {
    List<Account> accounts = AccountsDAO.getAllAccountsWithUsers();
    accountData.setAll(accounts);
}
```

```
    private void refreshAccountsView() {
        loadAccounts();
        handleSearch();
    }

    private void handleSearch() {
        String searchText = searchAccountField.getText() == null ? "" : searchAccountField.getText().trim().toLowerCase();
        if (searchText.isEmpty()) {
            accountTableView.setItems(accountData);
            return;
        }
        /*
         * Here I used lambda expression. matchesSearch is applied to each account instance from accountData
         * If matchesSearch returns true, account will be added to accountTableView
         */
        accountTableView.setItems(accountData.filtered(account -> matchesSearch(account, searchText)));
    }

    private boolean matchesSearch(Account account, String searchText) {
        if (searchText.isEmpty()) {
            return true;
        }

        return containsText(account.getName(), searchText)
                || containsText(account.getType(), searchText)
                || containsText(account.getUsers(), searchText);
    }

    private boolean containsText(String value, String searchText) {
        return value != null && value.toLowerCase().contains(searchText);
    }
}
```

CategoriesController.java

```
package timur.popov.financemanager.controller.tabs;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import timur.popov.financemanager.MainWindow;
import timur.popov.financemanager.controller.dialogs.CreatingCategoryController;
import timur.popov.financemanager.dao.CategoriesDAO;
import timur.popov.financemanager.model.Category;
import timur.popov.financemanager.DialogUtil;

import java.io.IOException;

/**
 * Controller that handles all interaction with Category tab
 */
public class CategoriesController {
    @FXML private TextField searchCategoryField;
    @FXML private Button searchButton;
    @FXML private Button addNewCategoryButton;

    @FXML private TableView<Category> categoryTableView;
    @FXML private TableColumn<Category, String> categoryTypeColumn;
    @FXML private TableColumn<Category, String> categoryNameColumn;

    private final ObservableList<Category> categoryData = FXCollections.observableArrayList();

    @FXML
    public void initialize() {
        //Binding columns to fields in Category class
        categoryTypeColumn.setCellValueFactory(new PropertyValueFactory<>("type"));
        categoryNameColumn.setCellValueFactory(new PropertyValueFactory<>("name"));
```

```
    loadCategoriesFromDatabase();

    categoryTableView.setItems(categoryData);

    //By clicking twice on specific raw, showEditDeleteDialog appears
    categoryTableView.setRowFactory(tv -> {
        TableRow<Category> row = new TableRow<>();
        row.setOnMouseClicked(event -> {
            if (event.getClickCount() == 2 && !row.isEmpty()) {
                showEditDeleteDialog(row.getItem());
            }
        });
        return row;
    });


    searchButton.setOnAction(event -> handleSearch());
    addNewCategoryButton.setOnAction(event -> openCreateCategoryDialog());
}

private void openCreateCategoryDialog() {
    try {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource("view/dialogs/create-category-dialog.fxml"));
        Dialog<CreatingCategoryController.CategoryData> dialog = new Dialog<>();

        dialog.setDialogPane(fxmlLoader.load());
        dialog.setTitle("Create new Category");

        //Each dialog has own controller which handle entered data
        CreatingCategoryController controller = fxmlLoader.getController();

        dialog.setResultConverter(dialogButton -> {
            if (dialogButton == ButtonType.APPLY) {
                // method buildResult() creates and returns AccountData instance with entered data
                return controller.buildResult();
            } else if (dialogButton == ButtonType.CLOSE) {
                // return empty data when operation is canceled
                return new CreatingCategoryController.CategoryData(null, null);
            }
            return null;
        });
        // Retrieving CategoryData from dialog
        CreatingCategoryController.CategoryData data = DialogUtil.showUntilPresent(dialog).get();

        //If dialog was closed before completing, type = null -> method will stop
        if(data.name() == null) {
            return;
        }

        boolean saved = CategoriesDAO.createCategory(data.name(), data.type());
        if (saved) {
            refreshCategoriesView();
        } else {
            DialogUtil.showError("Error", "Could not save category");
        }

    } catch (IOException e) {
        System.err.println("Could not find create-category-dialog.fxml.");
        e.printStackTrace();
    }
}

private void loadCategoriesFromDatabase() {
    categoryData.setAll(CategoriesDAO.getAllCategories());
}

private void showEditDeleteDialog(Category category) {
    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
    alert.setTitle("Category Actions");
    alert.setHeaderText("Edit or Delete Category");
```

```
    alert.setContentText("Choose an action for: " + category.getName());

    ButtonType editButton = new ButtonType("Edit");
    ButtonType deleteButton = new ButtonType("Delete", ButtonBar.ButtonData.CANCEL_CLOSE);
    ButtonType cancelButton = new ButtonType("Cancel", ButtonBar.ButtonData.NO);
    alert.getButtonTypes().setAll(editButton, deleteButton, cancelButton);

    alert.showAndWait().ifPresent(type -> {
        if (type == editButton) {
            handleEdit(category);
        } else if (type == deleteButton) {
            handleDelete(category);
        }
    });
}

private void handleEdit(Category category) {
    try {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource("view/dialogs/create-category-dialog.fxml"));
        Dialog<CreatingCategoryController.CategoryData> dialog = new Dialog<>();
        dialog.setDialogPane(fxmlLoader.load());
        dialog.setTitle("Edit Category");

        CreatingCategoryController controller = fxmlLoader.getController();
        controller.setInitialData(category.getName(), category.getType());

        // Opening dialog for editing account data
        dialog.setResultConverter(dialogButton -> {
            if (dialogButton == ButtonType.APPLY) {
                return controller.buildResult();
            } else if (dialogButton == ButtonType.CLOSE) {
                return new CreatingCategoryController.CategoryData(null, null);
            }
            return null;
        });

        // Retrieving CategoryData from dialog
        CreatingCategoryController.CategoryData data = DialogUtil.showUntilPresent(dialog).get();
        if (data.name() == null) {
            return;
        }

        boolean updated = CategoriesDAO.updateCategory(category.getCategoryId(), data.name(), data.type());
        if (updated) {
            refreshCategoriesView();
        } else {
            DialogUtil.showError("Error", "Could not update category.");
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

private void handleDelete(Category category) {
    Alert confirm = new Alert(Alert.AlertType.CONFIRMATION);
    confirm.setTitle("Delete Category");
    confirm.setHeaderText("Delete selected category?");
    confirm.setContentText(category.getName());

    confirm.showAndWait().ifPresent(buttonType -> {
        if (buttonType != ButtonType.OK) {
            return;
        }

        boolean deleted = CategoriesDAO.deleteCategory(category.getCategoryId());
        if (deleted) {
            refreshCategoriesView();
        } else {
            DialogUtil.showError("Error", "Could not delete category. It may be used in transactions.");
        }
    });
```

```
    }

    private void refreshCategoriesView() {
        loadCategoriesFromDatabase();
        handleSearch();
    }

    private void handleSearch() {
        String searchText = searchCategoryField.getText().toLowerCase();
        if (searchText.isEmpty()) {
            categoryTableView.setItems(categoryData);
        } else {
            /*
             * Here I used lambda expression. each category instance in categoryData checks that it contains
             * searched text. If it returns true, category will be added to categoryTableView
             */
            ObservableList<Category> filteredList = categoryData.filtered(category ->
                    category.getName().toLowerCase().contains(searchText) ||
                            category.getType().toLowerCase().contains(searchText)
            );
            categoryTableView.setItems(filteredList);
        }
    }
}
```

TransactionsTableConntroller.java

```
package timur.popov.financemanager.controller.tabs;

import javafx.fxml.FXML;
import javafx.scene.control.TableCell;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.Tooltip;
import javafx.scene.control.cell.PropertyValueFactory;
import timur.popov.financemanager.model.Transaction;

/**
 * Transaction table controller that is used in multiple places
 */
public class TransactionsTableController {
    @FXML private TableView<Transaction> transactionsTableView;
    @FXML private TableColumn<Transaction, String> typeColumn;
    @FXML private TableColumn<Transaction, String> categoryColumn;
    @FXML private TableColumn<Transaction, Double> amountColumn;
    @FXML private TableColumn<Transaction, String> dateColumn;
    @FXML private TableColumn<Transaction, String> userColumn;
    @FXML private TableColumn<Transaction, String> accountColumn;
    @FXML private TableColumn<Transaction, String> descriptionColumn;

    @FXML
    public void initialize() {
        setupTableColumns();
    }

    private void setupTableColumns() {
        //Binding table columns to fields in Transaction class
        typeColumn.setCellValueFactory(new PropertyValueFactory<>("type"));
        categoryColumn.setCellValueFactory(new PropertyValueFactory<>("category"));
        amountColumn.setCellValueFactory(new PropertyValueFactory<>("amount"));
        dateColumn.setCellValueFactory(new PropertyValueFactory<>("date"));
        userColumn.setCellValueFactory(new PropertyValueFactory<>("user"));

        accountColumn.setCellValueFactory(new PropertyValueFactory<>("account"));
        descriptionColumn.setCellValueFactory(new PropertyValueFactory<>("description"));


        // implementation of tooltip that keeps long descriptions readable when user points cursor at description field
        descriptionColumn.setCellFactory(column -> new TableCell<Transaction, String>() {
            @Override
```

```
        protected void updateItem(String item, boolean empty) {
            super.updateItem(item, empty);

            if (empty || item == null) {
                setText(null);
                setTooltip(null);
            } else {
                setText(item);


                Tooltip tooltip = new Tooltip(item);
                tooltip.setWrapText(true);
                tooltip.setMaxWidth(200);

                setTooltip(tooltip);
            }
        }
    });
  }
}
```

TransactionsController.java

```
package timur.popov.financemanager.controller.tabs;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.geometry.Insets;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import timur.popov.financemanager.dao.TransactionsDAO;
import timur.popov.financemanager.model.Transaction;
import timur.popov.financemanager.DialogUtil;

import java.time.LocalDate;
import java.util.List;

/**
 * Controller that handles all interaction with Transaction tab
 */
public class TransactionsController {
  @FXML private Button searchButton;
  @FXML private TableView<Transaction> transactionsTable;
  @FXML private ComboBox<String> accountFilterComboBox;
  @FXML private ComboBox<String> categoryFilterComboBox;
  @FXML private ComboBox<String> typeFilterComboBox;
  @FXML private ComboBox<String> userFilterComboBox;
  @FXML private DatePicker dateFromPicker;
  @FXML private DatePicker dateToPicker;
  @FXML private Spinner<Double> amountFromSpinner;
  @FXML private Spinner<Double> amountToSpinner;
  @FXML private TextField searchTransactionsField;


  @FXML
  public void initialize() {
    // Load values to filters
    setupFilterComboBox(accountFilterComboBox, "All Accounts", TransactionsDAO.getColumnData("Account", "Name"));
    setupFilterComboBox(userFilterComboBox, "All Users", TransactionsDAO.getColumnData("User", "Name"));
    setupFilterComboBox(categoryFilterComboBox, "All Categories", TransactionsDAO.getColumnData("Category", "Name"));
    typeFilterComboBox.setItems(FXCollections.observableArrayList("All Types", "Income", "Expense"));
    typeFilterComboBox.getSelectionModel().selectFirst();

    dateFromPicker.setValue(null);
    dateToPicker.setValue(null);
    setupRowActions();
    setupAmountSpinners();
    handleSearch();
```

```
    searchButton.setOnAction(event -> handleSearch());
}

private void setupFilterComboBox(ComboBox<String> comboBox, String firstItem, List<String> items) {
    ObservableList<String> options = FXCollections.observableArrayList(firstItem);
    options.addAll(items);
    comboBox.setItems(options);
    comboBox.getSelectionModel().selectFirst();
}

private void setupRowActions() {
    transactionsTable.setRowFactory(tv -> {
        TableRow<Transaction> row = new TableRow<>();
        row.setOnMouseClicked(event -> {
            if (event.getClickCount() == 2 && !row.isEmpty()) {
                //By clicking twice on specific raw, showEditDeleteDialog appears
                showEditDeleteDialog(row.getItem());
            }
        });
        return row;
    });
}

private void setupAmountSpinners() {
    amountFromSpinner.setValueFactory(new SpinnerValueFactory.DoubleSpinnerValueFactory(0.0, 100000.0, 0.0, 10.0));
    amountToSpinner.setValueFactory(new SpinnerValueFactory.DoubleSpinnerValueFactory(0.0, 100000.0, 100000.0, 10.0));
    amountFromSpinner.setEditable(true);
    amountToSpinner.setEditable(true);
}

private void showEditDeleteDialog(Transaction transaction) {

    Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
    alert.setTitle("Transaction Actions");
    alert.setHeaderText("Edit or Delete Transaction");
    alert.setContentText("Choose an action for: " + transaction.getDescription());

    ButtonType editButton = new ButtonType("Edit");
    ButtonType deleteButton = new ButtonType("Delete", ButtonBar.ButtonData.CANCEL_CLOSE);
    ButtonType cancelButton = new ButtonType("Cancel", ButtonBar.ButtonData.NO);

    alert.getButtonTypes().setAll(editButton, deleteButton, cancelButton);

    alert.showAndWait().ifPresent(type -> {
        if (type == editButton) {
            handleEdit(transaction);
        } else if (type == deleteButton) {
            handleDelete(transaction);
        }
    });
}

private void handleEdit(Transaction transaction) {
    // Creating dialog and adding fields to it
    Dialog<ButtonType> dialog = new Dialog<>();
    dialog.setTitle("Edit Transaction");

    TextField amountField = new TextField(String.valueOf(transaction.getAmount()));
    amountField.setPromptText("Amount");

    TextArea descriptionArea = new TextArea(transaction.getDescription() == null ? "" : transaction.getDescription());
    descriptionArea.setPromptText("Description");
    descriptionArea.setPrefRowCount(3);

    VBox content = new VBox(10,
            new Label("Amount"), amountField,
            new Label("Description"), descriptionArea
    );
    content.setPadding(new Insets(10));
```

```
    dialog.getDialogPane().setContent(content);
    dialog.getDialogPane().getButtonTypes().addAll(ButtonType.APPLY, ButtonType.CANCEL);

    dialog.showAndWait().ifPresent(buttonType -> {
        if (buttonType != ButtonType.APPLY) {
            return;
        }

        double newAmount;
        try {
            //converting string amount value from the field into that double
            newAmount = Double.parseDouble(amountField.getText());
        } catch (NumberFormatException e) {
            DialogUtil.showError("Validation Error", "Amount must be a number.");
            return;
        }
        // Checking that amount is a positive number
        if (newAmount <= 0) {
            DialogUtil.showError("Validation Error", "Amount must be positive.");
            return;
        }

        boolean updated = TransactionsDAO.updateTransactionBasic(
                transaction.getId(),
                newAmount,
                descriptionArea.getText()
        );

        if (updated) {
            //refreshing table with actual data
            handleSearch();
        } else {
            DialogUtil.showError("Error", "Could not update transaction.");
        }
    });
}

private void handleDelete(Transaction transaction) {
    Alert confirm = new Alert(Alert.AlertType.CONFIRMATION);
    confirm.setTitle("Delete Transaction");
    confirm.setHeaderText("Delete selected transaction?");
    confirm.setContentText(transaction.getDescription());

    confirm.showAndWait().ifPresent(buttonType -> {
        if (buttonType != ButtonType.OK) {
            return;
        }

        boolean deleted = TransactionsDAO.deleteTransaction(transaction.getId());
        if (deleted) {
            //refreshing table with actual data
            handleSearch();
        } else {
            DialogUtil.showError("Error", "Could not delete transaction.");
        }
    });
}


@FXML
public void handleSearch() {
    //Retrieving data from filters
    String searchText = searchTransactionsField.getText();

    String selectedType = normalizeFilterValue(typeFilterComboBox, "All Types");
    String selectedUser = normalizeFilterValue(userFilterComboBox, "All Users");
    String selectedAccount = normalizeFilterValue(accountFilterComboBox, "All Accounts");
    String selectedCategory = normalizeFilterValue(categoryFilterComboBox, "All Categories");

    LocalDate fromDate = dateFromPicker.getValue();
    LocalDate toDate = dateToPicker.getValue();
```

```
        Double minAmount = (Double) amountFromSpinner.getValue();
        Double maxAmount = (Double) amountToSpinner.getValue();


        //Getting spinner values that appear when program starts(default values)
        Double minBound = ((SpinnerValueFactory.DoubleSpinnerValueFactory) amountFromSpinner.getValueFactory()).getMin();
        Double maxBound = ((SpinnerValueFactory.DoubleSpinnerValueFactory) amountToSpinner.getValueFactory()).getMax();

        // If default values are not changed, we assume that this filter is not used, so we set it to null
        if (minAmount != null && minAmount.equals(minBound)) {
            minAmount = null;
        }
        if (maxAmount != null && maxAmount.equals(maxBound)) {
            maxAmount = null;
        }

        List<Transaction> results = TransactionsDAO.searchTransactions(
                searchText, selectedType, selectedUser, selectedAccount, selectedCategory, fromDate, toDate, minAmount, maxAmount
        );

        transactionsTable.setItems(FXCollections.observableArrayList(results));
    }

    private String normalizeFilterValue(ComboBox<String> comboBox, String allValue) {
        String value = comboBox.getValue();
        return (value == null || value.equals(allValue)) ? null : value;
    }

}
```

CreatingAccountController.java

```
package timur.popov.financemanager.controller.dialogs;

import javafx.fxml.FXML;
import javafx.scene.control.*;
import timur.popov.financemanager.DialogUtil;
import timur.popov.financemanager.dao.UserDAO;
import timur.popov.financemanager.model.User;

import java.util.ArrayList;
import java.util.List;

/**
 * Dialog controller for account creation/editing.
 */
public class CreatingAccountController {
    @FXML private ComboBox<String> typeComboBox;
    @FXML private TextField accountNameField;
    @FXML private ListView<User> usersListView;

    @FXML
    public void initialize() {
        //adding options to ComboBox
        typeComboBox.getItems().addAll("Debit", "Credit", "Savings", "Cash", "Investment");

        // SelectionMode.MULTIPLE will allow multiple users to be selected.
        usersListView.getSelectionModel().setSelectionMode(SelectionMode.MULTIPLE);

        /*
         * By default, ListView will display users using item.toString()
         * In this case, users will be displayed like User@76e15ffd which is not user-friendly
         * This code override updateItem method, so that ListView will use item.getName() to display users
         */
        usersListView.setCellFactory(list -> new ListCell<>() {
            @Override
            protected void updateItem(User item, boolean empty) {
                super.updateItem(item, empty);
                setText(empty || item == null ? "" : item.getName());
            }
```

```
        });

        usersListView.getItems().setAll(UserDAO.getAllUsers());
    }

    public AccountData buildResult() {
        //getting entered data from dialog
        String selectedType = typeComboBox.getValue();
        String accountName = accountNameField.getText() == null ? "" : accountNameField.getText().trim();
        List<User> selectedUsers = usersListView.getSelectionModel().getSelectedItems();

        if (selectedType == null || selectedType.isBlank()) {
            DialogUtil.showError("Validation Error", "Type must be selected");
            return null;
        }

        if (accountName.isBlank()) {
            DialogUtil.showError("Validation Error", "Account name cannot be empty");
            return null;
        }

        if (selectedUsers == null || selectedUsers.isEmpty()) {
            DialogUtil.showError("Validation Error", "Select at least one user");
            return null;
        }

        List<Integer> userIds = new ArrayList<>();
        //filling out list with user IDs
        for (User user : selectedUsers) {
            userIds.add(user.getUserId());
        }

        return new AccountData(selectedType, accountName, userIds);
    }

    /*
     * This method is needed if user wants to edit an account
     * It sets values for the specific account into dialog
     */
    public void setInitialData(String type, String accountName, List<Integer> selectedUserIds) {
        typeComboBox.setValue(type);
        accountNameField.setText(accountName);

        if (selectedUserIds == null || selectedUserIds.isEmpty()) {
            return;
        }

        usersListView.getSelectionModel().clearSelection();
        for (int i = 0; i < usersListView.getItems().size(); i++) {
            User user = usersListView.getItems().get(i);
            if (selectedUserIds.contains(user.getUserId())) {
                usersListView.getSelectionModel().select(i);
            }
        }
    }

    public record AccountData(String type, String accountName, List<Integer> userIds) {}
}
```

CreatingCategoryController.java

```
package timur.popov.financemanager.controller.dialogs;


import javafx.fxml.FXML;
import javafx.scene.control.*;
import timur.popov.financemanager.DialogUtil;

/**
 * Dialog controller for category creation/editing.
```

```
 */
public class CreatingCategoryController {

    @FXML public TextField nameField;
    @FXML public ComboBox<String> typeComboBox;
    @FXML public TextArea descriptionArea;

    public CategoryData buildResult() {
        String categoryName = nameField.getText() == null ? "" : nameField.getText().trim();
        String categoryType = (typeComboBox.getValue() == null) ? null : typeComboBox.getValue().toString();

        if (categoryName.isEmpty()) {
            DialogUtil.showError("Validation Error", "Category name cannot be empty");
            return null;
        }

        if (categoryType == null) {
            DialogUtil.showError("Validation Error", "Category type must be selected");
            return null;
        }

        return new CategoryData(categoryName, categoryType);
    }

    /*
     * This method is needed if user wants to edit an account
     * It sets values for the specific account into dialog
     */
    public void setInitialData(String name, String type) {
        nameField.setText(name);
        typeComboBox.setValue(type);
    }


    @FXML
    public void initialize() {
        typeComboBox.getItems().addAll(
                "Income",
                "Expense"
        );
    }

    public record CategoryData(String name, String type) {}
}
```

LogInController.java

```
package timur.popov.financemanager.controller.dialogs;

import javafx.fxml.FXML;
import javafx.scene.Node;
import javafx.scene.control.*;
import timur.popov.financemanager.PasswordHashUtil;
import timur.popov.financemanager.model.User;
import timur.popov.financemanager.DialogUtil;

/**
 * Login dialog controller that handle data entered in log-in window
 */
public class LogInController {

    @FXML private DialogPane loginDialogPane;
    @FXML private Label userNameLabel;
    @FXML private PasswordField passwordField;

    private User user;

    public void setUser(User user) {
        this.user = user;
        userNameLabel.setText(user.getName());
```

```
        Node applyNode = loginDialogPane.lookupButton(ButtonType.APPLY);
        ((Button) applyNode).setText("Login");

        Node closeNode = loginDialogPane.lookupButton(ButtonType.CLOSE);
        ((Button) closeNode).setText("Cancel");
    }

    public Boolean buildResult() {
        String entered = passwordField.getText();

        if (entered.isEmpty()) {
            DialogUtil.showError("Login Error", "Password cannot be empty.");
            return null;
        }

        //checking that entered password is correct
        if (!PasswordHashUtil.matches(entered, user.getPassword())) {
            // if password doesn't match, it clears password field and dialog continues waiting for correct data
            DialogUtil.showError("Login Error", "Incorrect password. Please try again.");
            passwordField.clear();
            return null;
        }

        return true;
    }
}
```

UserRegistrationController.java

```
package timur.popov.financemanager.controller.dialogs;

import javafx.fxml.FXML;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import timur.popov.financemanager.DialogUtil;

/**
 * Dialog controller for user registration.
 */
public class UserRegistrationController {
    @FXML public TextField nameField;
    @FXML public PasswordField passwordField;

    public RegistrationData buildResult() {
        String name = nameField.getText() == null ? "" : nameField.getText().trim();
        String password = passwordField.getText() == null ? "" : passwordField.getText();

        if (name.isEmpty()) {
            DialogUtil.showError("Validation Error", "Name cannot be empty.");
            return null;
        }

        if (password.isEmpty()) {
            DialogUtil.showError("Validation Error", "Password cannot be empty.");
            return null;
        }

        return new RegistrationData(name, password);
    }

    public record RegistrationData(String name, String password) {
    }
}
```

ManageUsersController.java

```
package timur.popov.financemanager.controller.dialogs;
```

```
import javafx.collections.FXCollections;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.control.*;
import javafx.scene.layout.HBox;
import javafx.scene.layout.Priority;
import javafx.scene.layout.Region;
import timur.popov.financemanager.MainWindow;
import timur.popov.financemanager.PasswordHashUtil;
import timur.popov.financemanager.dao.UserDAO;
import timur.popov.financemanager.model.User;
import timur.popov.financemanager.DialogUtil;

import java.io.IOException;
import java.util.Optional;

/**
 * Dialog controller for user administration actions.
 */
public class ManageUsersController {
    @FXML
    private ListView<User> usersListView;

    @FXML
    public void initialize() {
        //This code is needed to display user names in the ListView and add edit/delete buttons for each user.
        usersListView.setCellFactory(listView -> new ListCell<>() {
            // It updates each row in ListView
            @Override
            protected void updateItem(User user, boolean empty) {
                super.updateItem(user, empty);
                //Handling empty rows
                if (empty || user == null) {
                    setText(null);
                    setGraphic(null);
                    return;
                }
                //Declaring buttons and creating a grid
                Label nameLabel = new Label(user.getName());

                Button editButton = new Button("Edit name");
                editButton.setOnAction(event -> editUserName(user));

                Button deleteButton = new Button("Delete user");
                deleteButton.setOnAction(event -> deleteUser(user));

                Region spacer = new Region();
                HBox.setHgrow(spacer, Priority.ALWAYS);

                HBox row = new HBox(10, nameLabel, spacer, editButton, deleteButton);
                setText(null);
                setGraphic(row);
            }
        });

        loadUsers();
    }

    private void loadUsers() {
        usersListView.setItems(FXCollections.observableArrayList(UserDAO.getAllUsers()));
    }

    private void editUserName(User user) {
        if (!confirmPassword(user)) {
            return;
        }

        TextInputDialog dialog = new TextInputDialog(user.getName());
        dialog.setTitle("Edit Name");
        dialog.setHeaderText("Change user name");
        dialog.setContentText("New name:");
```

```
    Optional<String> result = dialog.showAndWait();
    if (result.isEmpty()) {
        return;
    }

    String newName = result.get().trim();
    if (newName.isEmpty()) {
        DialogUtil.showError("Error", "Name cannot be empty.");
        return;
    }

    if (UserDAO.updateUserName(user.getUserId(), newName)) {
        loadUsers();
    } else {
        DialogUtil.showError("Error", "Could not update user name.");
    }
}

private void deleteUser(User user) {
    if (!confirmPassword(user)) {
        return;
    }

    Alert confirmation = new Alert(Alert.AlertType.CONFIRMATION,
            "Delete user '" + user.getName() + "'?", ButtonType.OK, ButtonType.CANCEL);
    confirmation.setTitle("Delete User");
    confirmation.setHeaderText("Please confirm");

     /*
      * If user press Close, dialog will be closed
      * If user will close dialog, confirmation.showAndWait() will be ButtonType.CANCEL and method will be over
      */
    if (confirmation.showAndWait().orElse(ButtonType.CANCEL) != ButtonType.OK) {
        return;
    }

    if (UserDAO.deleteUser(user.getUserId())) {
        loadUsers();
    } else {
        DialogUtil.showError("Error", "Could not delete user. This user may have linked records.");
    }
}

private boolean confirmPassword(User user) {
    Dialog<String> dialog = new Dialog<>();
    dialog.setTitle("Password Required");
    dialog.setHeaderText("Enter password for " + user.getName());

    PasswordField passwordField = new PasswordField();
    passwordField.setPromptText("Password");
    dialog.getDialogPane().setContent(passwordField);
    dialog.getDialogPane().getButtonTypes().addAll(ButtonType.OK, ButtonType.CANCEL);

    dialog.setResultConverter(buttonType -> buttonType == ButtonType.OK ? passwordField.getText() : null);
    Optional<String> result = dialog.showAndWait();

    // Password gate is an authorization check before destructive operations.
    return result.isPresent() && PasswordHashUtil.matches(result.get(), user.getPassword());
}

@FXML
private void openAddUserDialog() {
    try {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource("view/dialogs/user-registration-dialog.fxml"));
        Dialog<UserRegistrationController.RegistrationData> dialog = new Dialog<>();
        dialog.setDialogPane(fxmlLoader.load());
        dialog.setTitle("Add User");
        dialog.getDialogPane().getStylesheets().add(MainWindow.class.getResource("style.css").toExternalForm());

        UserRegistrationController controller = fxmlLoader.getController();
```

```
        dialog.setResultConverter(buttonType -> {
            if (buttonType == ButtonType.APPLY) {
                return controller.buildResult();
            }
            return null;
        });

        Optional<UserRegistrationController.RegistrationData> result = dialog.showAndWait();

        if (result.isPresent()) {
            String hashed = PasswordHashUtil.hash(result.get().password());
            boolean saved = UserDAO.createUser(result.get().name(), hashed);
            if (saved) {
                loadUsers();
                DialogUtil.showInfo("User Created", "New user was added.");
            } else {
                DialogUtil.showError("Error", "Could not save user.");
            }
        }
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
  }
}
```

QuickActionController.java

```
package timur.popov.financemanager.controller.dialogs;

import javafx.fxml.FXML;
import javafx.scene.control.*;
import timur.popov.financemanager.UserSession;
import timur.popov.financemanager.dao.TransactionsDAO;
import timur.popov.financemanager.model.User;
import timur.popov.financemanager.DialogUtil;

import java.util.List;

/**
 * Shared dialog controller used by both add income and add expense dialogs.
 * It is possible because income and expense dialogs have same fields
 */
public class QuickActionController {
    @FXML public TextField amountField;
    @FXML public ComboBox<String> accountComboBox;
    @FXML public ComboBox<String> categoryComboBox;
    @FXML public TextArea descriptionArea;
    private String transactionType;

    public void setTransactionType(String transactionType) {
        this.transactionType = transactionType;
        loadOptionsFromDatabase();
    }

    private void loadOptionsFromDatabase() {
        User currentUser = UserSession.getInstance().getCurrentUser();
        if (currentUser == null) {
            return;
        }

        List<String> accounts = TransactionsDAO.getAccountsForUser(currentUser.getUserId());
        //Getting all available accounts for current user
        accountComboBox.getItems().setAll(accounts);
        //Getting all available categories for current user
        List<String> categories = TransactionsDAO.getCategoriesByType(transactionType);
        categoryComboBox.getItems().setAll(categories);
    }

    public EnteredData buildResult() {
```

```
        String amountField = this.amountField.getText();
        String descriptionField = this.descriptionArea.getText();
        String account = accountComboBox.getValue();
        String category = categoryComboBox.getValue();


        double amount;
        if(amountField.isEmpty()){
            DialogUtil.showError("Validation Error", "Amount cannot be empty.");
            return null;
        } else {
            try {
                amount = Double.parseDouble(amountField);
            } catch (NumberFormatException e) {
                DialogUtil.showError("Validation Error", "Amount must be a number");
                return null;
            }
        }

        if (amount <= 0) {
            DialogUtil.showError("Validation Error", "Amount must be a positive value");
            return null;
        }

        if (account == null) {
            DialogUtil.showError("Validation Error", "Account must be selected");
            return null;
        }

        if (category == null) {
            DialogUtil.showError("Validation Error", "Category must be selected");
            return null;
        }

        return new EnteredData(amount, account, category, descriptionField);
    }


    @FXML
    public void initialize() {
        loadOptionsFromDatabase();
    }

    public record EnteredData(double amount, String account, String category, String description) {}
}
```

MainWindowController.java

```
package timur.popov.financemanager.controller;

import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.control.*;
import javafx.scene.layout.BorderPane;
import javafx.util.StringConverter;
import timur.popov.financemanager.MainWindow;
import timur.popov.financemanager.UserSession;
import timur.popov.financemanager.controller.dialogs.LogInController;
import timur.popov.financemanager.controller.dialogs.QuickActionController;
import timur.popov.financemanager.dao.TransactionsDAO;
import timur.popov.financemanager.dao.UserDAO;
import timur.popov.financemanager.model.User;
import timur.popov.financemanager.DialogUtil;

import java.io.IOException;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
```

```
 * Primary MVC controller that coordinates the work of main window
 */
public class MainWindowController {
    @FXML
    public ComboBox<User> userSwitcherComboBox;
    @FXML
    private BorderPane mainBorderPane;

    private User lastLogged = null;

    private String currentSection = "dashboard";

    @FXML
    public void initialize() {

        //Change the format of displayed users(from User@76e15ffd to user name)
        userSwitcherComboBox.setConverter(new StringConverter<User>() {
            @Override
            public String toString(User user) {
                return user != null ? user.getName() : "";
            }
            @Override
            public User fromString(String string) { return null; }
        });


        List<User> users = UserDAO.getAllUsers();
        userSwitcherComboBox.getItems().setAll(users);


        // Start locked so the user must authenticate before interacting with app content.
        userSwitcherComboBox.getSelectionModel().clearSelection();
        userSwitcherComboBox.setPromptText("Select User");
        UserSession.getInstance().setCurrentUser(null);
        lastLogged = null;
        setAppLocked(true);
        DialogUtil.showInfo("Login Required", "Please choose a user and log in to continue.");

        //handling user switch and call dialog to log in
        userSwitcherComboBox.setOnAction(event -> {
            User selected = userSwitcherComboBox.getValue();
            if (selected == null) return;

            if (lastLogged != null && selected.getUserId() == lastLogged.getUserId()) return;

            boolean success = showLoginDialog(selected);
            if (success) {
                lastLogged = selected;
                UserSession.getInstance().setCurrentUser(selected);
                setAppLocked(false);
            } else {
                setAppLocked(true);
                lastLogged = null;
            }
        });
    }

    //switches tab on the main window
    public void SwitchSectionOnMainWindow(ActionEvent actionEvent) {
        if (!ensureAuthenticated()) {
            return;
        }
        // Regex extracts button id from ActionEvent string
        Pattern pattern = Pattern.compile("id=(\\w+)");

        //actionEvent.toString() = javafx.event.ActionEvent[source=Button[id=transactionsButton, styleClass=button sidebar-
button]'Transactions']
        Matcher matcher = pattern.matcher(actionEvent.toString());
        String result;
        try {
            if (matcher.find()) {
```

```
            result = matcher.group(1).replaceFirst("Button$", "");
            currentSection = result;
        } else {
            throw new Exception("wrong name of Button");
        }

        loadSection(result);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }


}
@FXML
private void handleManageUsers() {
    if (!ensureAuthenticated()) {
        return;
    }

    try {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource("view/dialogs/manage-users-dialog.fxml"));
        Dialog<ButtonType> dialog = new Dialog<>();
        dialog.setDialogPane(fxmlLoader.load());
        dialog.setTitle("Manage Users");
        dialog.getDialogPane().getStylesheets().add(MainWindow.class.getResource("style.css").toExternalForm());

        dialog.showAndWait();
        refreshUserSwitcher();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

@FXML
public void handleAddIncome(ActionEvent actionEvent) throws IOException {
    if (!ensureAuthenticated()) {
        return;
    }

    openQuickActionDialog(
            "view/dialogs/add-income-dialog.fxml",
            "Add Income",
            "Income",
            "Income transaction has been saved.",
            "Could not save income transaction."
    );
}
@FXML
public void handleAddExpense(ActionEvent actionEvent) throws IOException {
    if (!ensureAuthenticated()) {
        return;
    }

    openQuickActionDialog(
            "view/dialogs/add-expense-dialog.fxml",
            "Add Expanse",
            "Expense",
            "Expense transaction has been saved.",
            "Could not save expense transaction."
    );
}

private void openQuickActionDialog(String resourcePath, String dialogTitle, String transactionType,
                                   String successMessage, String errorMessage) throws IOException {
    FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource(resourcePath));
    Dialog<QuickActionController.EnteredData> dialog = new Dialog<>();

    dialog.setDialogPane(fxmlLoader.load());
    dialog.setTitle(dialogTitle);

    QuickActionController controller = fxmlLoader.getController();
```

```
    controller.setTransactionType(transactionType);

    dialog.setResultConverter(dialogButton -> {
        if (dialogButton == ButtonType.APPLY) {
            return controller.buildResult();
        }
        if (dialogButton == ButtonType.CLOSE) {
            // If user cancel operation of adding new transaction, the empty object will be returned.
            return new QuickActionController.EnteredData(-1, null, null, null);
        }
        return null;
    });

    QuickActionController.EnteredData data = DialogUtil.showUntilPresent(dialog).get();

    //check that object is not empty
    if (data.amount() == -1) {
        return;
    }

    boolean saved = TransactionsDAO.createTransaction(
            UserSession.getInstance().getCurrentUser().getUserId(),
            data.account(),
            data.category(),
            transactionType,
            data.amount(),
            data.description()
    );

    if (saved) {
        refreshVisibleTransactionData();
        DialogUtil.showInfo("Success", successMessage);
    } else {
        DialogUtil.showError("Error", errorMessage);
    }
}

//After adding new transaction this method updates tables in dashboard and transactions sections
private void refreshVisibleTransactionData() {
    if ("transactions".equals(currentSection) || "dashboard".equals(currentSection)) {
        loadSection(currentSection);
    }
}

private void loadSection(String sectionName) {
    try {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource(String.format("view/%s.fxml", sectionName)));
        mainBorderPane.setCenter(fxmlLoader.load());
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}

private boolean showLoginDialog(User user) {
    try {
        FXMLLoader fxmlLoader = new FXMLLoader(MainWindow.class.getResource("view/dialogs/log-in-dialog.fxml"));
        Dialog<Boolean> dialog = new Dialog<>();
        dialog.setDialogPane(fxmlLoader.load());
        dialog.setTitle("Login");

        LogInController controller = fxmlLoader.getController();
        controller.setUser(user);

        dialog.setResultConverter(buttonType -> {
            if (buttonType == ButtonType.APPLY) {
                return controller.buildResult();
            }
            return false;
        });

        return DialogUtil.showUntilPresent(dialog).get();
```

```
        } catch (IOException e) {
            e.printStackTrace();
            return false;
        }
    }

    private void refreshUserSwitcher() {
        User current = UserSession.getInstance().getCurrentUser();
        Integer currentUserId = current != null ? current.getUserId() : null;

        List<User> users = UserDAO.getAllUsers();
        userSwitcherComboBox.getItems().setAll(users);

        User selected = null;
        if (currentUserId != null) {
            for (User user : users) {
                if (user.getUserId() == currentUserId) {
                    selected = user;
                    break;
                }
            }
        }

        userSwitcherComboBox.setValue(selected);

        UserSession.getInstance().setCurrentUser(selected);
        lastLogged = selected;
        setAppLocked(false);
    }

    private boolean ensureAuthenticated() {
        // Guard clause pattern: central place that blocks protected actions in this controller.
        if (UserSession.getInstance().getCurrentUser() != null) {
            return true;
        }

        DialogUtil.showInfo("Login Required", "Please choose a user and log in to continue.");
        return false;
    }

    private void setAppLocked(boolean locked) {
        if (mainBorderPane.getCenter() != null) {
            mainBorderPane.getCenter().setDisable(locked);
        }
    }

}
```

main-window.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.ComboBox?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.Separator?>
<?import javafx.scene.layout.BorderPane?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.Region?>
<?import javafx.scene.layout.VBox?>

<BorderPane fx:id="mainBorderPane" prefHeight="768.0" prefWidth="1024.0" styleClass="border-pane"
xmlns="http://javafx.com/javafx/20.0.2" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="timur.popov.financemanager.controller.MainWindowController">
   <left>
      <VBox fx:id="leftSidebar" prefWidth="240.0" styleClass="left-sidebar" BorderPane.alignment="TOP_LEFT">
         <children>
            <HBox>
               <padding>
```

```
                <Insets bottom="10.0" left="15.0" right="15.0" top="10.0" />
              </padding>
              <children>
                <ComboBox fx:id="userSwitcherComboBox" prefHeight="40.0" prefWidth="180.0" promptText="Select User"
HBox.hgrow="ALWAYS" />
              </children>
            </HBox>
            <Separator styleClass="sidebar-separator" />

            <VBox spacing="5.0">
              <padding>
                <Insets bottom="15.0" left="15.0" right="15.0" top="15.0" />
              </padding>
              <children>
                <Button fx:id="dashboardButton" maxWidth="1.7976931348623157E308" mnemonicParsing="false"
onAction="#SwitchSectionOnMainWindow" styleClass="sidebar-button" text="Dashboard" />
                <Button fx:id="transactionsButton" maxWidth="1.7976931348623157E308" mnemonicParsing="false"
onAction="#SwitchSectionOnMainWindow" styleClass="sidebar-button" text="Transactions" />
                <Button fx:id="accountsButton" maxWidth="1.7976931348623157E308" mnemonicParsing="false"
onAction="#SwitchSectionOnMainWindow" styleClass="sidebar-button" text="Accounts" />
                <Button fx:id="categoriesButton" maxWidth="1.7976931348623157E308" mnemonicParsing="false"
onAction="#SwitchSectionOnMainWindow" styleClass="sidebar-button" text="Categories" />
              </children>
            </VBox>

            <Separator styleClass="sidebar-separator" />

            <VBox spacing="10.0">
              <padding>
                <Insets bottom="15.0" left="15.0" right="15.0" top="15.0" />
              </padding>
              <children>
                <Label text="Quick Actions" styleClass="quick-actions-label" />
                <Button fx:id="addIncomeButton" maxWidth="1.7976931348623157E308" mnemonicParsing="false"
onAction="#handleAddIncome" styleClass="quick-action-button" text="Add Income" />
                <Button fx:id="addExpenseButton" maxWidth="1.7976931348623157E308" mnemonicParsing="false"
onAction="#handleAddExpense" styleClass="quick-action-button" text="Add Expense"  />
              </children>
            </VBox>

            <Region VBox.vgrow="ALWAYS" />

            <VBox spacing="5.0">
              <padding>
                <Insets bottom="15.0" left="15.0" right="15.0" top="10.0" />
              </padding>
              <children>
                <Separator styleClass="sidebar-separator" />
                <Button fx:id="manageUsersButton" maxWidth="1.7976931348623157E308" mnemonicParsing="false"
onAction="#handleManageUsers" styleClass="sidebar-button" text="Manage Users" />
              </children>
            </VBox>
          </children>
        </VBox>
    </left>
    <!--Tabs will be imported here from different files-->
    <center>
      <fx:include fx:id="contentArea" source="dashboard.fxml" />
    </center>
</BorderPane>
```

dashboard.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.Separator?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.VBox?>
```

```
<?import javafx.scene.text.Font?>

<VBox prefHeight="728.0" prefWidth="824.0" spacing="20.0" style="-fx-background-color: #f7f7f7;"
xmlns="http://javafx.com/javafx/20.0.2" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="timur.popov.financemanager.controller.tabs.DashboardController">
   <padding>
      <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
   </padding>
   <children>
      <Label text="Dashboard" textFill="#333333">
         <font>
            <Font name="System Bold" size="24.0" />
         </font>
      </Label>
      <HBox alignment="CENTER" spacing="25.0">
         <children>
            <VBox alignment="CENTER" prefHeight="140.0" prefWidth="240.0" style="-fx-background-color: #e6ffe6; -fx-background-radius:
15; -fx-border-color: #d0edd0; -fx-border-radius: 15; -fx-border-width: 1;">
               <padding>
                  <Insets bottom="15.0" left="15.0" right="15.0" top="15.0" />
               </padding>
               <children>
                  <Label text="Total Monthly Income" textAlignment="CENTER" textFill="#4CAF50">
                     <font>
                        <Font size="16.0" />
                     </font>
                  </Label>
                  <Label fx:id="totalIncomeLabel" text="\$3,200.00" textFill="#2E7D32">
                     <font>
                        <Font name="System Bold" size="28.0" />
                     </font>
                  </Label>
               </children>
            </VBox>

            <VBox alignment="CENTER" prefHeight="140.0" prefWidth="240.0" style="-fx-background-color: #ffe6e6; -fx-background-radius:
15; -fx-border-color: #edd0d0; -fx-border-radius: 15; -fx-border-width: 1;">
               <padding>
                  <Insets bottom="15.0" left="15.0" right="15.0" top="15.0" />
               </padding>
               <children>
                  <Label text="Total Monthly Expenses" textAlignment="CENTER" textFill="#F44336">
                     <font>
                        <Font size="16.0" />
                     </font>
                  </Label>
                  <Label fx:id="totalExpenseLabel" text="\$1,850.50" textFill="#C62828">
                     <font>
                        <Font name="System Bold" size="28.0" />
                     </font>
                  </Label>
               </children>
            </VBox>

         </children>
         <VBox.margin>
            <Insets bottom="10.0" />
         </VBox.margin>
      </HBox>

      <Separator prefHeight="1.0" prefWidth="200.0" style="-fx-background-color: #cccccc;">
         <VBox.margin>
            <Insets left="20.0" right="20.0" />
         </VBox.margin>
      </Separator>

      <Label text="Recent Transactions" textFill="#333333">
         <font>
            <Font name="System Bold" size="18.0" />
         </font>
         <VBox.margin>
```

```
            <Insets left="20.0" top="10.0" />
          </VBox.margin>
        </Label>

        <fx:include fx:id="recentTransactionsTable"
                    source="transactionsTable.fxml"
                    VBox.vgrow="ALWAYS"
                    mouseTransparent="true"
                    focusTraversable="false">

        </fx:include>
    </children>
</VBox>
```

categories.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TableColumn?>
<?import javafx.scene.control.TableView?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.Pane?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<VBox alignment="TOP_CENTER" spacing="20.0" style="-fx-background-color: #f7f7f7;" xmlns="http://javafx.com/javafx/20.0.2"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="timur.popov.financemanager.controller.tabs.CategoriesController">
    <padding>
        <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
    </padding>

    <Label text="Categories" textFill="#333333">
        <font>
            <Font name="System Bold" size="24.0" />
        </font>
    </Label>

    <HBox alignment="CENTER_LEFT" spacing="10.0">
        <children>
            <TextField fx:id="searchCategoryField" promptText="Search categories..." HBox.hgrow="ALWAYS" />
            <Button fx:id="searchButton" mnemonicParsing="false" text="Search" style="-fx-background-color: #6c757d; -fx-text-fill: white;" />
            <Pane HBox.hgrow="ALWAYS" />
            <Button fx:id="addNewCategoryButton" mnemonicParsing="false" style="-fx-background-color: #28a745; -fx-text-fill: white; -fx-
font-weight: bold;" text="Add New Category" />
        </children>
    </HBox>

    <TableView fx:id="categoryTableView" VBox.vgrow="ALWAYS">
        <columns>
            <TableColumn fx:id="categoryTypeColumn" prefWidth="150.0" text="Type" />
            <TableColumn fx:id="categoryNameColumn" prefWidth="200.0" text="Category Name" />
        </columns>
        <columnResizePolicy>
            <TableView fx:constant="CONSTRAINED_RESIZE_POLICY" />
        </columnResizePolicy>
    </TableView>
</VBox>
```

accounts.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
```

```
<?import javafx.scene.control.TableColumn?>
<?import javafx.scene.control.TableView?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.Pane?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<VBox alignment="TOP_CENTER" spacing="20.0" style="-fx-background-color: #f7f7f7;" xmlns="http://javafx.com/javafx/20.0.2"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="timur.popov.financemanager.controller.tabs.AccountsController">
   <padding>
      <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
   </padding>

   <Label text="Accounts" textFill="#333333">
      <font>
         <Font name="System Bold" size="24.0" />
      </font>
   </Label>

   <HBox alignment="CENTER_LEFT" spacing="10.0">
      <children>
         <TextField fx:id="searchAccountField" promptText="Search accounts..." HBox.hgrow="ALWAYS" />
         <Button fx:id="searchButton" mnemonicParsing="false" text="Search" style="-fx-background-color: #6c757d; -fx-text-fill: white;" />
         <Pane HBox.hgrow="ALWAYS" />
         <Button fx:id="addNewAccountButton" mnemonicParsing="false" style="-fx-background-color: #28a745; -fx-text-fill: white; -fx-
font-weight: bold;" text="Create New Account" />
      </children>
   </HBox>

   <TableView fx:id="accountTableView" VBox.vgrow="ALWAYS">
      <columns>
         <TableColumn fx:id="accountTypeColumn" prefWidth="170.0" text="Account Type" />
         <TableColumn fx:id="accountNameColumn" prefWidth="220.0" text="Name" />
         <TableColumn fx:id="accountUsersColumn" prefWidth="300.0" text="Users" />
      </columns>
      <columnResizePolicy>
         <TableView fx:constant="CONSTRAINED_RESIZE_POLICY" />
      </columnResizePolicy>
   </TableView>
</VBox>
```

transactionsTable.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.scene.text.*?>

<VBox prefHeight="728.0" prefWidth="824.0" spacing="20.0" style="-fx-background-color: #f7f7f7;"
xmlns="http://javafx.com/javafx/20.0.2" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="timur.popov.financemanager.controller.tabs.TransactionsController">
   <padding>
      <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
   </padding>
   <children>
      <Label text="Transactions Overview" textFill="#333333">
         <font>
            <Font name="System Bold" size="24.0" />
         </font>
      </Label>

      <VBox spacing="15.0">
         <children>
            <HBox alignment="BOTTOM_LEFT" spacing="15.0">
               <children>
                  <VBox spacing="5.0">
                     <children>
                        <Label text="Type" textFill="#555555" />
                        <ComboBox fx:id="typeFilterComboBox" prefWidth="150.0" promptText="Type" />
```

```
            </children>
          </VBox>

          <VBox spacing="5.0">
            <children>
              <Label text="Filters" textFill="#555555" />
              <HBox spacing="5.0">
                <children>
                  <ComboBox fx:id="userFilterComboBox" prefWidth="110.0" promptText="User" />
                  <ComboBox fx:id="accountFilterComboBox" prefWidth="110.0" promptText="Account" />
                  <ComboBox fx:id="categoryFilterComboBox" prefWidth="130.0" promptText="Category" />
                </children>
              </HBox>
            </children>
          </VBox>

          <VBox spacing="5.0">
            <children>
              <Label text="Date Range" textFill="#555555" />
              <HBox spacing="5.0">
                <children>
                  <DatePicker fx:id="dateFromPicker" prefWidth="130.0" promptText="From" />
                  <DatePicker fx:id="dateToPicker" prefWidth="130.0" promptText="To" />
                </children>
              </HBox>
            </children>
          </VBox>

          <VBox spacing="5.0">
            <children>
              <Label text="Amount Range" textFill="#555555" />
              <HBox spacing="5.0">
                <children>
                  <Spinner fx:id="amountFromSpinner" prefWidth="140.0" promptText="From" />
                  <Spinner fx:id="amountToSpinner" prefWidth="140.0" promptText="To" />
                </children>
              </HBox>
            </children>
          </VBox>
        </children>
      </HBox>

      <VBox spacing="5.0">
        <children>
          <Label text="Search by Description" textFill="#555555" />
          <HBox spacing="10.0" alignment="CENTER_LEFT">
            <children>
              <TextField fx:id="searchTransactionsField"
                         prefHeight="35.0"
                         prefWidth="600.0"
                         HBox.hgrow="ALWAYS"
                         promptText="Search transactions by description" />

              <Button fx:id="searchButton"
                      mnemonicParsing="false"
                      prefHeight="35.0"
                      prefWidth="100.0"
                      style="-fx-background-color: #28a745; -fx-text-fill: white; -fx-font-weight: bold;"
                      text="Search"/>
            </children>
          </HBox>
        </children>
      </VBox>
    </children>
  </VBox>

  <fx:include fx:id="transactionsTable"
          source="transactionsTable.fxml"
          VBox.vgrow="ALWAYS">

  </fx:include>
```

```
  </children>
</VBox>
```

transactionsTable.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.scene.control.TableColumn?>
<?import javafx.scene.control.TableView?>

<TableView xmlns:fx="http://javafx.com/fxml/1"
        xmlns="http://javafx.com/javafx/20.0.2"
        fx:id="transactionsTableView" fx:controller="timur.popov.financemanager.controller.tabs.TransactionsTableController">
   <columns>
      <TableColumn fx:id="typeColumn" prefWidth="75.0" text="Type" resizable="false" reorderable="false" />
      <TableColumn fx:id="categoryColumn" prefWidth="100.0" text="Category" resizable="false" reorderable="false" />
      <TableColumn fx:id="amountColumn" prefWidth="100.0" text="Amount" resizable="false" reorderable="false" />
      <TableColumn fx:id="dateColumn" prefWidth="120.0" text="Date" resizable="false" reorderable="false" />
      <TableColumn fx:id="userColumn" prefWidth="100.0" text="User" resizable="false" reorderable="false" />
      <TableColumn fx:id="accountColumn" prefWidth="120.0" text="Account" resizable="false" reorderable="false" />
      <TableColumn fx:id="descriptionColumn" prefWidth="130.0" text="Description" resizable="false" reorderable="false" />
   </columns>
   <columnResizePolicy>
      <TableView fx:constant="CONSTRAINED_RESIZE_POLICY" />
   </columnResizePolicy>
</TableView>
```

add-expense-dialog.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<DialogPane fx:id="addExpenseDialogPane" headerText="Add New Expense Transaction" prefWidth="400.0"
xmlns="http://javafx.com/javafx/20.0.2" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="timur.popov.financemanager.controller.dialogs.QuickActionController">
   <content>
      <VBox spacing="15.0">
         <children>
            <VBox spacing="5.0">
               <Label text="Account" />
               <ComboBox fx:id="accountComboBox" maxWidth="1.7976931348623157E308" promptText="Select Account"
styleClass="combo-box" />
            </VBox>

            <VBox spacing="5.0">
               <Label text="Amount ($)" />
               <TextField fx:id="amountField" promptText="e.g., 75.50" styleClass="text-field" />
            </VBox>

            <VBox spacing="5.0">
               <Label text="Category" />
               <ComboBox fx:id="categoryComboBox" maxWidth="1.7976931348623157E308" promptText="Select expense category"
styleClass="combo-box" />
            </VBox>

            <VBox spacing="5.0">
               <Label text="Description (Optional)" />
               <TextArea fx:id="descriptionArea" prefHeight="80.0" promptText="e.g., Groceries, Rent, Utilities" styleClass="text-area" />
            </VBox>
         </children>
         <padding>
            <Insets bottom="15.0" left="20.0" right="20.0" top="15.0" />
         </padding>
      </VBox>
   </content>
```

```
  <buttonTypes>
    <ButtonType fx:id="applyButton"  fx:constant="APPLY" />
    <ButtonType fx:id="closeButton" fx:constant="CLOSE" />
  </buttonTypes>
</DialogPane>
```

add-income-dialog.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.*?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>

<DialogPane fx:id="addIncomeDialogPane" headerText="Add New Income Transaction" prefWidth="400.0"
xmlns="http://javafx.com/javafx/20.0.2" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="timur.popov.financemanager.controller.dialogs.QuickActionController">
  <content>
    <VBox spacing="15.0">
      <children>
        <VBox spacing="5.0">
          <Label text="Account" />
          <ComboBox fx:id="accountComboBox" maxWidth="1.7976931348623157E308" promptText="Select Account"
styleClass="combo-box" />
        </VBox>

        <VBox spacing="5.0">
          <Label text="Amount ($)" />
          <TextField fx:id="amountField" promptText="e.g., 500.00" styleClass="text-field" />
        </VBox>

        <VBox spacing="5.0">
          <Label text="Source/Category" />
          <ComboBox fx:id="categoryComboBox" maxWidth="1.7976931348623157E308" promptText="Select income source"
styleClass="combo-box" />
        </VBox>

        <VBox spacing="5.0">
          <Label text="Description (Optional)" />
          <TextArea fx:id="descriptionArea" prefHeight="80.0" promptText="e.g., Monthly Salary, Freelance Payment" styleClass="text-
area" />
        </VBox>

      </children>
      <padding>
        <Insets bottom="15.0" left="20.0" right="20.0" top="15.0" />
      </padding>
    </VBox>
  </content>

  <buttonTypes>
    <ButtonType fx:id="applyButton" fx:constant="APPLY" />
    <ButtonType fx:id="closeButton" fx:constant="CLOSE" />
  </buttonTypes>
</DialogPane>
```

create-account-dialog.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.ButtonType?>
<?import javafx.scene.control.ComboBox?>
<?import javafx.scene.control.DialogPane?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.ListView?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.VBox?>
```

```
<DialogPane prefWidth="420.0" xmlns="http://javafx.com/javafx/20.0.2" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="timur.popov.financemanager.controller.dialogs.CreatingAccountController" headerText="Create New Account">
  <content>
    <VBox spacing="12.0">
      <children>
        <VBox spacing="5.0">
          <Label text="Type" />
          <ComboBox fx:id="typeComboBox" maxWidth="1.7976931348623157E308" promptText="Select account type"
styleClass="combo-box" />
        </VBox>

        <VBox spacing="5.0">
          <Label text="Account Name" />
          <TextField fx:id="accountNameField" promptText="Enter account name" styleClass="text-field" />
        </VBox>

        <VBox spacing="5.0">
          <Label text="Users (select one or more)" />
          <ListView fx:id="usersListView" prefHeight="170.0" />
        </VBox>
      </children>
      <padding>
        <Insets bottom="15.0" left="20.0" right="20.0" top="15.0" />
      </padding>
    </VBox>
  </content>

  <buttonTypes>
    <ButtonType fx:constant="APPLY" />
    <ButtonType fx:constant="CLOSE" />
  </buttonTypes>
</DialogPane>
```

create-category-dialog.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.ButtonType?>
<?import javafx.scene.control.ComboBox?>
<?import javafx.scene.control.DialogPane?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TextArea?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.VBox?>

<DialogPane fx:id="createCategoryDialogPane" headerText="Create New Category" prefWidth="400.0"
xmlns="http://javafx.com/javafx/20.0.2" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="timur.popov.financemanager.controller.dialogs.CreatingCategoryController">
  <content>
    <VBox spacing="15.0">
      <children>
        <VBox spacing="5.0">
          <Label text="Category Name" />
          <TextField fx:id="nameField" promptText="e.g., Groceries, Salary" styleClass="text-field"
maxWidth="1.7976931348623157E308" />
        </VBox>

        <VBox spacing="5.0">
          <Label text="Type" />
          <ComboBox fx:id="typeComboBox" maxWidth="1.7976931348623157E308" promptText="Select category type"
styleClass="combo-box" />
        </VBox>


        <VBox spacing="5.0">
          <Label text="Description (Optional)" />
          <TextArea fx:id="descriptionArea" prefHeight="80.0" promptText="Brief description of the category" styleClass="text-area"
maxWidth="1.7976931348623157E308" />
        </VBox>
      </children>
```

```
        <padding>
          <Insets bottom="15.0" left="20.0" right="20.0" top="15.0" />
        </padding>
      </VBox>
    </content>

    <buttonTypes>
      <ButtonType fx:id="applyButton" fx:constant="APPLY" />
      <ButtonType fx:id="closeButton" fx:constant="CLOSE" />
    </buttonTypes>
</DialogPane>
```

log-in-dialog.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.ButtonType?>
<?import javafx.scene.control.DialogPane?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.PasswordField?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<DialogPane fx:id="loginDialogPane" headerText="Login" prefWidth="350.0"
        xmlns="http://javafx.com/javafx/20.0.2" xmlns:fx="http://javafx.com/fxml/1"
        fx:controller="timur.popov.financemanager.controller.dialogs.LogInController">
    <content>
      <VBox spacing="15.0">
        <padding>
          <Insets bottom="15.0" left="20.0" right="20.0" top="15.0" />
        </padding>
        <children>
          <!-- User greeting -->
          <VBox spacing="3.0">
            <Label text="Welcome back," />
            <Label fx:id="userNameLabel" text="" />
          </VBox>

          <!-- Password input -->
          <VBox spacing="5.0">
            <Label text="Password" />
            <PasswordField fx:id="passwordField"
                       promptText="Enter your password"
                       styleClass="text-field"
                       maxWidth="1.7976931348623157E308" />
          </VBox>
        </children>
      </VBox>
    </content>

    <buttonTypes>
      <ButtonType fx:id="applyButton" fx:constant="APPLY" />
      <ButtonType fx:id="closeButton" fx:constant="CLOSE" />
    </buttonTypes>
</DialogPane>
```

manage-users-dialog.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.ButtonType?>
<?import javafx.scene.control.DialogPane?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.ListView?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.Region?>
```

```
<?import javafx.scene.layout.VBox?>

<DialogPane fx:id="manageUsersDialogPane"
        headerText="Manage Users"
        prefWidth="560.0"
        xmlns="http://javafx.com/javafx/20.0.2"
        xmlns:fx="http://javafx.com/fxml/1"
        fx:controller="timur.popov.financemanager.controller.dialogs.ManageUsersController">
   <content>
     <VBox spacing="15.0">
       <padding>
         <Insets bottom="15.0" left="20.0" right="20.0" top="15.0" />
       </padding>
       <children>
         <Label text="Users" />
         <ListView fx:id="usersListView" prefHeight="320.0" VBox.vgrow="ALWAYS">
           <placeholder>
             <Label text="No users found." />
           </placeholder>
         </ListView>
         <HBox alignment="CENTER_RIGHT" spacing="10.0">
           <children>
             <Region HBox.hgrow="ALWAYS" />
             <Button fx:id="addUserButton"
                 mnemonicParsing="false"
                 onAction="#openAddUserDialog"
                 text="Add User"
                 styleClass="quick-action-button" />
           </children>
         </HBox>
       </children>
     </VBox>
   </content>

   <buttonTypes>
     <ButtonType fx:id="closeButton" fx:constant="CLOSE" />
   </buttonTypes>
</DialogPane>
```

user-registration-dialog.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.ButtonType?>
<?import javafx.scene.control.DialogPane?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.PasswordField?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.VBox?>

<DialogPane fx:id="registerDialogPane" headerText="New User Registration" prefWidth="350.0" xmlns="http://javafx.com/javafx/20.0.2"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="timur.popov.financemanager.controller.dialogs.UserRegistrationController">
   <content>
     <VBox spacing="15.0">
       <children>
         <VBox spacing="5.0">
           <Label text="Name" />
           <TextField fx:id="nameField" promptText="Enter name" styleClass="text-field" maxWidth="1.7976931348623157E308" />
         </VBox>

         <VBox spacing="5.0">
           <Label text="Password" />
           <PasswordField fx:id="passwordField" promptText="Enter password" styleClass="text-field"
maxWidth="1.7976931348623157E308" />
         </VBox>
       </children>
       <padding>
         <Insets bottom="15.0" left="20.0" right="20.0" top="15.0" />
       </padding>
     </VBox>
```

```
  </content>

  <buttonTypes>
    <ButtonType fx:id="applyButton" fx:constant="APPLY" />
    <ButtonType fx:id="closeButton" fx:constant="CLOSE" />
  </buttonTypes>
</DialogPane>
```

style.css

```
/*Global Styles*/
.root {
  -fx-font-family: "Segoe UI", "Arial", sans-serif;
  -fx-font-size: 13px;
  -fx-background-color: #f7f7f7;
}

/* Main Window (BorderPane and Sidebar) */
.border-pane {
  -fx-background-color: #f7f7f7;
}

.left-sidebar {
  -fx-background-color: #3f4a3e;
  -fx-padding: 10px 0;
}

.app-title-container {
  -fx-alignment: CENTER_LEFT;
  -fx-padding: 0 20px;
  -fx-min-height: 60px;
}

.app-title {
  -fx-font-size: 20px;
  -fx-font-weight: bold;
  -fx-text-fill: white;
}

.sidebar-separator {
  -fx-background-color: #556654;
  -fx-min-height: 1px;
  -fx-max-height: 1px;
}

.sidebar-button {
  -fx-background-color: transparent;
  -fx-text-fill: #e0e0e0;
  -fx-font-size: 15px;
  -fx-alignment: CENTER_LEFT;
  -fx-padding: 8px 15px;
  -fx-background-radius: 5px;
  -fx-pref-height: 35px;
  -fx-cursor: hand;
}

.sidebar-button:hover {
  -fx-background-color: #556654;
  -fx-text-fill: white;
}

.sidebar-button:focused,
.sidebar-button.active {
  -fx-background-color: #6a8069;
  -fx-text-fill: white;
  -fx-font-weight: bold;
}

.quick-actions-label {
  -fx-font-size: 13px;
```

```
    -fx-font-weight: bold;
    -fx-text-fill: #a0a0a0;
}

.quick-action-button,
.login-button {
    -fx-background-color: #28a745;
    -fx-text-fill: white;
    -fx-font-weight: bold;
    -fx-cursor: hand;
}

.quick-action-button:hover,
.login-button:hover {
    -fx-background-color: #218838;
}

.quick-action-button {
    -fx-font-size: 13px;
    -fx-background-radius: 8px;
    -fx-padding: 8px 12px;
    -fx-alignment: CENTER_LEFT;
}
```

creating_tables.sql

All tables that I have in database are listed in this file. When program starts If there is no database file, it will be created by using this sql file

```
CREATE TABLE IF NOT EXISTS User
(
    UserID          INTEGER PRIMARY KEY AUTOINCREMENT,
    Name            varchar(255) NOT NULL,
    Password        varchar(255) NOT NULL,
    RegistrationDate date        NOT NULL
);

CREATE TABLE IF NOT EXISTS Account
(
    AccountID     INTEGER PRIMARY KEY AUTOINCREMENT,
    Name          varchar(255) NOT NULL,
    Type          varchar(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS UserAccount
(
    UserID    int,
    AccountID int,
    PRIMARY KEY (UserID, AccountID),
    FOREIGN KEY (UserID) REFERENCES User (UserID)
        ON DELETE CASCADE,
    FOREIGN KEY (AccountID) REFERENCES Account (AccountID)
        ON DELETE CASCADE
);


CREATE TABLE IF NOT EXISTS Category
(
    CategoryID INTEGER PRIMARY KEY AUTOINCREMENT,
    Name       varchar(255) NOT NULL,
    Type       varchar(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS Transactions
(
    TransactionID INTEGER PRIMARY KEY AUTOINCREMENT,
    Amount        decimal NOT NULL,
    CreationDate  date    NOT NULL,
    Description   varchar(255),
    AccountID     int,
```

```
    UserID       int,
    CategoryID   int,
    FOREIGN KEY (AccountID) REFERENCES Account (AccountID),
    FOREIGN KEY (UserID) REFERENCES User (UserID),
    FOREIGN KEY (CategoryID) REFERENCES Category (CategoryID)
);
```

pom.xml

Pom file is a configuration file for Maven. It keeps all dependencies[23]

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>timur.popov</groupId>
    <artifactId>FinanceManager</artifactId>
    <version>1.0-SNAPSHOT</version>
    <name>FinanceManager</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <junit.version>5.12.1</junit.version>
        <java.version>20</java.version>
        <javafx.version>20.0.2</javafx.version>
        <main.class>timur.popov.financemanager.Launcher</main.class>
        <app.module>timur.popov.financemanager</app.module>
        <app.package.version>1.0.0</app.package.version>
    </properties>

    <dependencies>

        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>${javafx.version}</version>
        </dependency>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-fxml</artifactId>
            <version>${javafx.version}</version>
        </dependency>
        <!-- https://mvnrepository.com/artifact/org.xerial/sqlite-jdbc -->
        <dependency>
            <groupId>org.xerial</groupId>
            <artifactId>sqlite-jdbc</artifactId>
            <version>3.49.1.0</version>
        </dependency>

        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-api</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-engine</artifactId>
            <version>${junit.version}</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
```

[23] https://www.geeksforgeeks.org/advance-java/maven-pom/

```
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.13.0</version>
            <configuration>
              <source>${java.version}</source>
              <target>${java.version}</target>
            </configuration>
          </plugin>
          <plugin>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-maven-plugin</artifactId>
            <version>0.0.8</version>
            <executions>
              <execution>
                <!-- Default configuration for running with: mvn clean javafx:run -->
                <id>default-cli</id>
                <configuration>
                  <mainClass>${app.module}/${main.class}
                  </mainClass>
                  <launcher>app</launcher>
                  <jlinkZipName>app</jlinkZipName>
                  <jlinkImageName>app</jlinkImageName>
                  <noManPages>true</noManPages>
                  <stripDebug>true</stripDebug>
                  <noHeaderFiles>true</noHeaderFiles>
                </configuration>
              </execution>
            </executions>
          </plugin>
        </plugins>
    </build>
</project>
```

module-info.java

Java modules groups together related packages, classes, and resources, allowing them to be bundled and distributed as one unit[24] It was created by development environment when I created a project

```
module timur.popov.financemanager {
    requires javafx.controls;
    requires javafx.fxml;
    requires java.sql;
    requires javafx.graphics;
    requires jdk.jshell;
    requires javafx.base;


    opens timur.popov.financemanager to javafx.fxml;
    exports timur.popov.financemanager;
    exports timur.popov.financemanager.controller;
    opens timur.popov.financemanager.controller to javafx.fxml;
    exports timur.popov.financemanager.dao;
    opens timur.popov.financemanager.dao to javafx.fxml;
    exports timur.popov.financemanager.controller.tabs;
    opens timur.popov.financemanager.controller.tabs to javafx.fxml;
    opens timur.popov.financemanager.controller.dialogs to javafx.fxml;
    opens timur.popov.financemanager.model to javafx.base, javafx.fxml;
}
```

[24] https://medium.com/@reetesh043/java-modules-a-guide-for-beginners-c1dcd6a33a83

Testing

In this section, I will test that my program works properly. I recorded the video with a complete test. Also, I have created a table that will show what exactly I am testing. Next to each test, I will put indicator PASS/FALL and will add video time stamp for convenience.

I have uploaded the video with tests on YouTube that can be accessed via this link:

https://youtu.be/P9RUNvQMU_Y

Test Number	Sub-Success Criteria	PASS / FAIL	Video time stamp

Authentication & User Management			
1	System prevents access to main application tabs (Dashboard, Transactions) before a user successfully logs in.	PASS	0:04
2	Register a new user with valid inputs	PASS	0:52
3	Edit a user's name with wrong password	PASS	1:23
4	Log in using the correct password.	PASS	0:30
5	Attempts to log in with an incorrect password.	PASS	0:20
6	Edit a user's name with correct password	PASS	1:34
7	Delete a user	PASS	2:20
Account Management			
8	Create a new Account and assign it to multiple users.	PASS	3:24
9	Search for a specific Account	PASS	2:50
10	Edit an existing Account's name and type.	PASS	4:02
11	Delete an Account and check that it no longer appears in the lists of available accounts for user.	PASS	4:16
Category Management			
12	Create a new category.	PASS	9:07
13	Search for a specific category	PASS	8:50
14	Edit category’s name	PASS	9:22
15	Delete a Category and check that it is removed from the database.	PASS	9:40
Transaction Management			

(Quick Actions)			
16	Add new Income transaction	PASS	4:30
17	Add new Expense transaction	PASS	5:34
18	Attempt to add a transaction with a negative or non-numeric amount	PASS	4:35
19	Edit an existing transaction's amount and description in the transactions tab.	PASS	6:00
20	Delete a transaction and check that it disappears from table	PASS	6:30
Search & Dynamic Filtering			
21	Filter the transactions table by a specific account	PASS	7:04
22	Filter the transactions table by a specific category	PASS	7:19
23	Filter transactions using a specific date range	PASS	7:25
24	Filter transactions using by amount	PASS	7:45
25	Filter transactions by description via search field	PASS	8:16
26	Filter transactions by description via transaction type	PASS	6:45
27	Filter transactions by user	PASS	6:56
Dashboard & Aggregation Math			
28	Check that "Total Monthly Income" properly sums all income for the current month only.	PASS	5:05
29	Check that "Total Monthly Expenses" properly sums all expenses for the current month only.	PASS	5:40

30	Add a new transaction and check that Recent Transactions table update immediately as well as "Total Monthly Income" and "Total Monthly Expenses"	PASS	5:47

While testing my application, I found a minor bag. Some why after trying to log-in with wrong password, log-in window closes and you cannot log-in again for the same user. You have to switch to another user and when switch back to the user that you need. After this procedure, you will be able to log in again.

Evaluation

To conclude, I am satisfied with the result that I received. I managed to implement the basic functionality of the application. I enjoyed the process of developing and got crucial skills of coding that I will definitely use in my futures problems.

The hardest part of this project was managing request to database. I did my best to separate database request from other components. As a result, I got dao package that handle everything associated with database and queries. It lets me to make the structure of my project clear.

In analyses section, I created a table with objectives for this project. Here is the table that shows what objectives I met:

Objective	Success Criteria	PASS/FAIL
1. Secure User Authentication	a) Window with user and password fields	PASS
	b) Buttons “Cancel” and “Login”	PASS
	c) Applying hash function to password	PASS
	d) Comparing hash of entered password with hash stored in database	PASS
2. Hash Utility	a) Hash function algorithm that will return hash of the string	PASS
3. Database Implementation	a) Connecting to Database	PASS
	b) Checking that database contains all the tables needed	PASS
	c) Normalised form	PASS
	d) Closing connection after closing the app	PASS
4. CRUD operations with Database	a) Methods and queries for retrieving, updating, deleting and adding transactions	PASS
	b) Methods and queries for retrieving, updating, deleting and adding accounts	PASS
	c) Methods and queries for retrieving, updating, deleting and adding categories	PASS
	d) Methods and queries for retrieving, updating, deleting and adding users	PASS
5. Session Management	a) Tracking current user	PASS
	b) Change current user during switching	PASS
	c) Freezing app until user is not logged-in	PASS
6. Main window with tabs	a) Main window that shows current tab	PASS
	b) Sidebar with all tabs	PASS
	c) Quick action to add income/expense	PASS

	d) Switch user button	PASS
	e) Manage users button	PASS
7. Dashboard tab	a) Field that will show total monthly income and expense based on transactions	PASS
	b) Table with recent transactions	PASS
8. Transaction tab	a) Table with columns: type, category, amount, date, user, account, description	PASS
	b) Filters such as type, category, amount, date, user and account	PASS
	c) Search by description	PASS
9. Account tab	a) Table with all existing accounts	PASS
	b) Search field and corresponding button	PASS
	c) “Create new Account” button	PASS
10. Category tab	a) Table with all existing categories	PASS
	b) Search field and corresponding button	PASS
	c) “Create new Category” button	PASS
11. Adding new Transaction	a) Window for adding income/expense	PASS
	b) Account combobox	PASS
	c) Amount field	PASS
	d) Category combobox	PASS
	e) Description field	PASS
12. Deleting and Editing a Transaction	a) Window for deleting or editing transaction	PASS
	b) In case of editing, another window for changing amount or description	PASS
	a) SQL query is modified based on used filters	PASS

13. Filtering and searching transaction by parameters	b) Table shows only transactions that meet all criteria	PASS
14. Creating new Account	a) Opening separate window	PASS
	b) Combobox for account type	PASS
	c) Account name field	PASS
	d) List of users that can be added	PASS
15. Deleting and Editing an Account	a) Window for deleting or editing account	PASS
	b) In case of editing, another window for changing type, name or users	PASS
16. Searching needed Account	a) SQL query is modified based on used filters	PASS
	b) Table shows only accounts that meet all criteria	PASS
17. Creating new Category	a) Opening separate window	PASS
	b) Combobox for category type	PASS
	c) Category name field	PASS
	d) Description field	PASS
18. Deleting and Editing a Category	a) Window for deleting or editing category	PASS
	b) In case of editing, another window for changing type, name or description	PASS
19. Searching needed Category	a) SQL query is modified based on used filters	PASS
	b) Table shows only categories that meet all criteria	PASS
20. Switching between Users	a) List of users for switching	PASS
	b) Log-in window that will appear after switching users	PASS
	a) Window for adding new users	PASS

21. Registering new Users	b) Username and password field	PASS
	c) Hash function must be applied to password before storing in database	PASS
22. Deleting and Editing details of Users	a) Separate window for deleting or editing users	PASS
	b) In case of editing, separate window must appear with username field	PASS
23. Input Validation and Error Handling	a) The system rejects alphabetical characters entered into the numerical "Amount" field	PASS
	b) The system prevents submission of a transaction if mandatory fields (like Account or Category) are left blank, displaying an alert dialog instead	PASS

End User feedback

After completing development, I gave my application to a friend of mine. He used it for one month. After that month, he told me that he was generally happy with my application and found it useful for him. He specified that he liked simplicity of interface and ease of adding new transactions. Also, he enjoyed multi-user features that allowed his girlfriend to use it as well. Also, he told me what he didn't like. He said that app's window is too small and sometimes it is hard to see text on the screen. Also, he couldn't figure out how to edit transactions until I told him that it can be done by double clicking.

Potential improvements:

My app is already usable, however there are still some things that can be imrpoved.

1. Fixing minor bugs that appear in my application such as the bug with re-login for the same user that I mentioned before.
2. Expanding the functionality of my program. For example, adding new tabs with diagrams that will provide better understanding of how money is allocated
3. Make the interface more intuitive because now some people might be confused with the action needed for editing transactions, accounts and categories. Also, people with bad vision might want to have larger elements. I would add settings where user would be able to adjust the app to get the better user experience.
4. Finally, I would move database to the cloud, so users could use the app from different devices. It is quite a complicated task because it involves simultaneous

access to data that may lead to errors if access doesn’t set up in an appropriate way.

Conclusion

I am happy that I have completed this project and that I met all objectives that I specified at the beginning. There are some things that I didn’t add to my app because of lack of time. I enjoyed the process and on top of that I gain important experience of developing desktop app that I will definitely use in the future.